

Department of **Cybersecurity Technology Engineering**
Electrical Engineering Technical College
Middle Technical University



الحقيبة التعليمية لمادة أساسيات البرمجة للمرحلة الأولى
Programming Essentials

By
Dr. Loay Talib Al-Saffar*



*Education of Dr. Loay Al-Saffar:

Ph.D. in Computer Science, from the University of Potsdam, Department of Computer Science, Germany.

M.Sc. in Computer Science, from the University of Baghdad, Department of Computer Science, Iraq.

B.Sc. in Computer Science, from the University of Baghdad, Department of Computer Science, Iraq.

المحاضرة الأولى النظرية: مقدمة .. البرامج وأنواعها .. المكونات المادية للحاسبة وكيفية تمثيل البيانات.

Introduction

Think about some of the different ways that people use computers. In school, students use computers for tasks such as writing papers, searching for articles, sending email, and participating in online classes. At work, people use computers to analyse data, make presentations, conduct business transactions, communicate with customers and co-workers, control machines in manufacturing facilities, and many other things. At home, people use computers for tasks such as paying bills, shopping online, communicating with friends and family, and playing computer games. And don't forget that cell phones, car navigation systems, and many other devices are computers too. The uses of computers are almost limitless in our everyday lives.

Computers can do such a wide variety of things because they can be programmed. This means that computers are not designed to do just one job, but to do any job that their programs tell them to do. A *program* is a set of instructions that a computer follows to perform a task. For example, Figure 1 shows screens from two commonly used programs: Microsoft Word and PowerPoint.

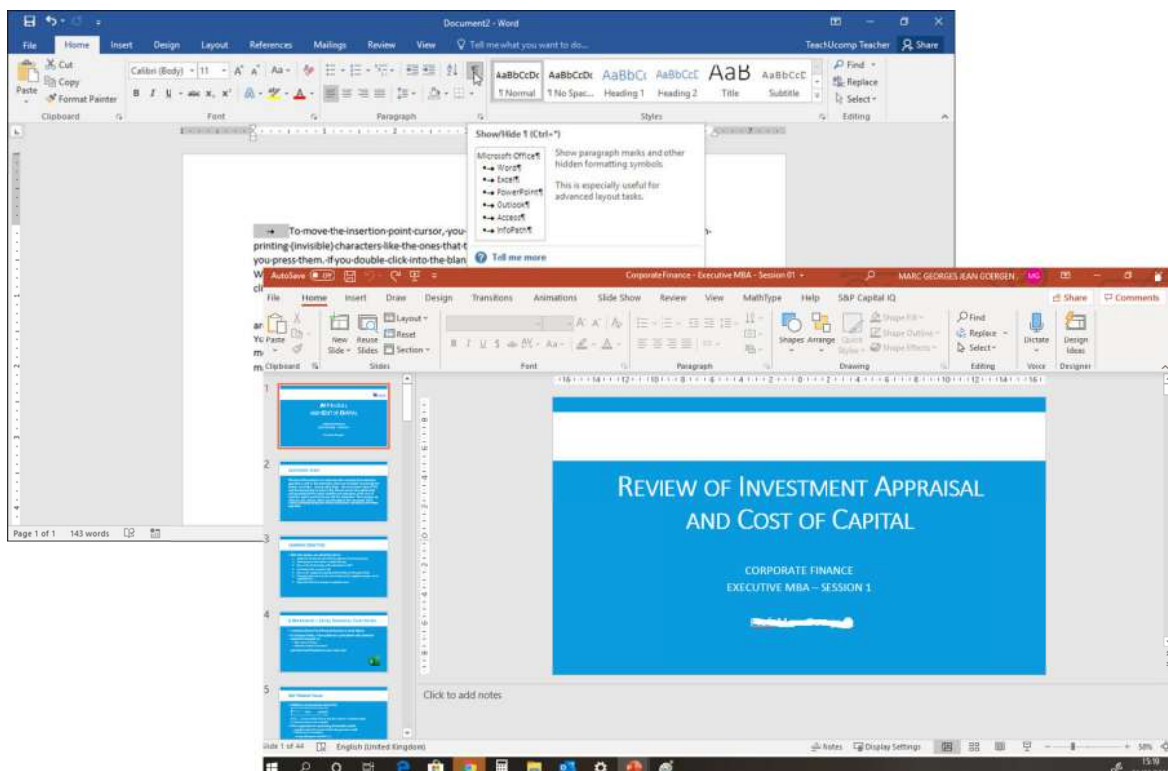


Figure 1: Two commonly used programs: Microsoft Word and PowerPoint.

Programs (Software)

Programs are commonly referred to as **software**. Software is essential to a computer because without software, a computer can do nothing. All of the software that we use to make our computers useful is created by individuals known as programmers or software developers. A **programmer**, or **software developer**, is a person with the training and skills necessary to design, create, and test computer programs. Computer programming is an exciting and rewarding career. Today, you will find programmers working in business, medicine, government, law enforcement, agriculture, academics, entertainment, and almost every other field.

The two major types of software are:

1• Application software: This software helps meet certain needs of users to perform specific tasks on a computer. Programs that make a computer useful for everyday tasks are known as *application software*. It includes MS Office (as shown in figure 1), paint, games, Web browsers, email programs, etc. It is usually written in a high-level language and is slower in performance.

2• System software: This software aids the user and the hardware to function and interact with each other. The programs that control and manage the basic operations of a computer are generally referred to as *system software*. It includes operating systems, drivers, etc. It is usually written in a low-level language and is faster in performance.

System software typically includes the following types of programs:

- **Operating Systems.** An *operating system* is the most fundamental set of programs on a computer. The operating system controls the internal operations of the computer's hardware, manages all of the devices connected to the computer, allows data to be saved to and retrieved from storage devices, and allows other programs to run on the computer. For example: Windows, Mac OS®, and Linux®.
- **Utility Programs.** A *utility program* performs a specialized task that enhances the computer's operation or safeguards data. Examples of utility programs are virus scanners, file compression programs, and data backup programs.

- **Software Development Tools.** *Software development tools* are the programs that programmers use to create, modify, and test software. Assemblers, compilers, and interpreters are examples of programs that fall into this category.

Hardware

The term *hardware* refers to all of the physical devices, or **components**, that a computer is made of. A computer is not one single device, but a system of devices that all work together. Like the different instruments in a symphony orchestra, each device in a computer plays its own part.

If you have ever shopped for a computer, you've probably seen sales literature listing components such as microprocessors, memory, disk drives, video displays, graphics cards, and so on. As shown in Figure 2, a typical computer system consists of the following major components:

- The central processing unit (CPU)
- Main memory
- Secondary storage devices
- Input devices
- Output devices

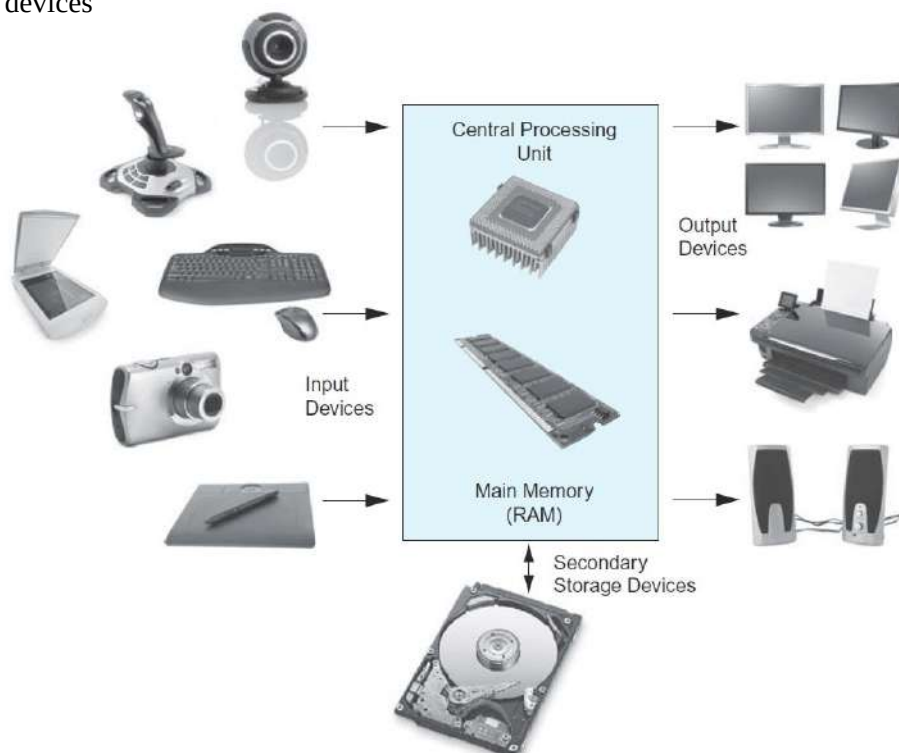


Figure 2: Typical components of a computer system.

The CPU

When a computer is performing the tasks that a program tells it to do, we say that the computer is **running** or **executing** the program. The *central processing unit*, or *CPU*, is the part of a computer that actually runs programs.

In the earliest computers, CPUs were huge devices made of electrical and mechanical components such as vacuum tubes and switches. Figure 3 shows such a device. The *ENIAC*, considered by many to be the world's first programmable electronic computer, was built in 1945 to calculate artillery ballistic tables for the U.S. Army. This machine, which was primarily one big CPU, was 8 feet tall, 100 feet long, and weighed 30 tons.

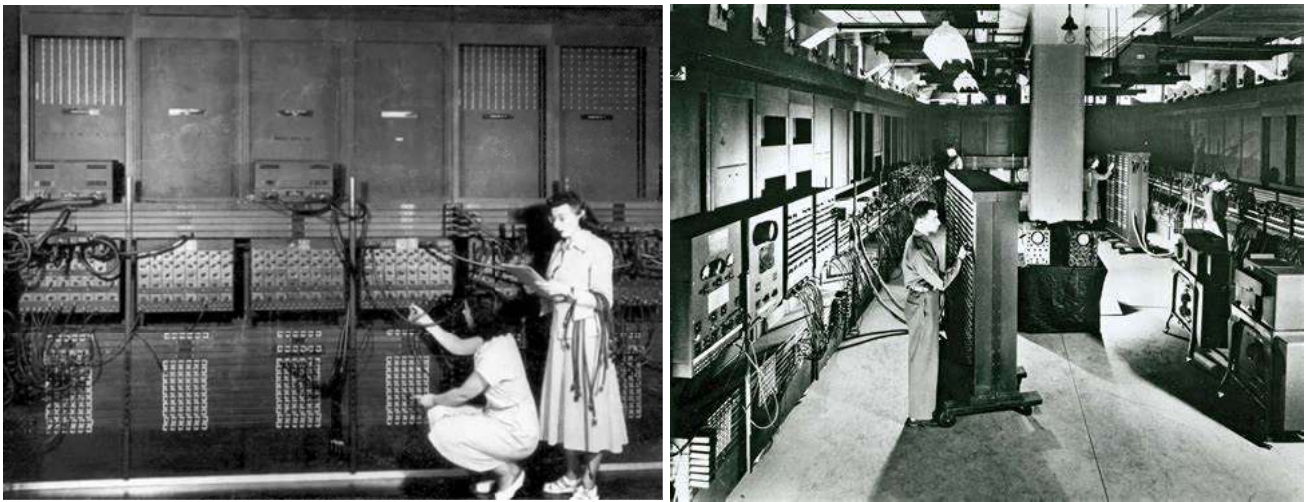


Figure 3 The ENIAC computer.

Today, CPUs are small chips known as **microprocessors**. Figure 4 shows a photo of a lab technician holding a modern-day microprocessor. In addition to being much smaller than the old electro-mechanical CPUs in early computers, microprocessors are also much more powerful.

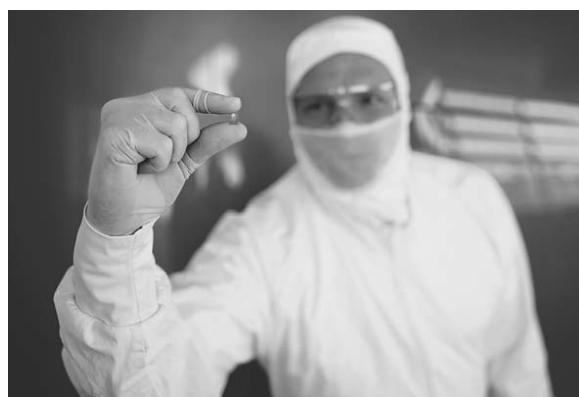


Figure 4 A lab technician holds a modern microprocessor

Main Memory

You can think of *main memory* as the computer's work area. This is where the computer stores a program while the program is running, as well as the data that the program is working with. For example, suppose you are using a word processing program to write an essay for one of your classes. While you do this, both the word processing program and the essay are stored in main memory.

Main memory is commonly known as ***random-access memory***, or ***RAM***. It is called this because the CPU is able to quickly access data stored at any random location in RAM. RAM is usually a ***volatile*** type of memory that is used only for temporary storage while a program is running. When the computer is turned off, the contents of RAM are erased. Inside your computer, RAM is stored in chips, similar to the ones shown in Figure 5.

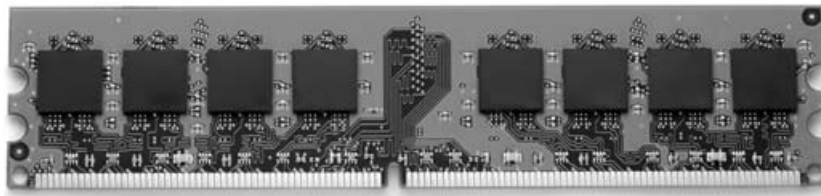


Figure 5 Memory chips.

Secondary Storage Devices

Secondary storage is a type of memory that can hold data for long periods of time, even when there is no power to the computer. Programs are normally stored in secondary memory and loaded into main memory as needed. Important data, such as word processing documents, payroll data, and inventory records, is saved to secondary storage as well. See Figure 6 for different types of storage devices.



Figure 6 Storage devices.

The most common type of secondary storage device is the disk drive. A *disk drive* stores data by magnetically encoding it onto a circular disk. Most computers have a disk drive mounted inside their case. External disk drives, which connect to one of the computer's communication ports, are also available. External disk drives can be used to create backup copies of important data or to move data to another computer.

In addition to external disk drives, many types of devices have been created for copying data, and for moving it to other computers. For many years floppy disk drives were popular. A *floppy disk drive* records data onto a small floppy disk, which can be removed from the drive. Floppy disks have many disadvantages, however. They hold only a small amount of data, are slow to access data, and are sometimes unreliable.

The use of floppy disk drives has declined dramatically in recent years, in favor of superior devices such as USB drives. USB drives are small devices that plug into the computer's USB (universal serial bus) port, and appear to the system as a disk drive. These drives do not actually contain a disk, however. They store data in a special type of memory known as flash memory. USB drives, which are also known as *memory sticks* and *flash drives*, are inexpensive, reliable, and small enough to be carried in your pocket.

Optical devices such as the *CD* (compact disc) and the *DVD* (digital versatile disc) are also popular for data storage. Data is not recorded magnetically on an optical disc, but is encoded as a series of pits on the disc surface. CD and DVD drives use a laser to detect the pits and thus read the encoded data.

Input Devices

Input is any data the computer collects from people and from other devices. The component that collects the data and sends it to the computer is called an *input device*. Common input devices are the keyboard, mouse, scanner, microphone, and digital camera. Disk drives and optical drives can also be considered input devices because programs and data are retrieved from them and loaded into the computer's memory.

Output Devices

Output is any data the computer produces for people or for other devices. It might be a sales report, a list of names, or a graphic image. The data is sent to an *output device*, which formats and presents it. Common output devices are video displays and printers. Disk drives and CD

recorders can also be considered output devices because the system sends data to them in order to be saved.

How Computers Store Data

A computer's memory is divided into tiny storage locations known as **bytes**. One byte is only enough memory to store a letter of the alphabet or a small number. In order to do anything meaningful, a computer has to have lots of bytes. Most computers today have millions, or even billions, of bytes of memory.

Each byte is divided into eight smaller storage locations known as **bits**. The term *bit* stands for **binary digit**. Computer scientists usually think of bits as tiny switches that can be either on or off. Bits aren't actual "switches," however, at least not in the conventional sense. In most computer systems, bits are tiny electrical components that can hold either a positive or a negative charge. Computer scientists think of a positive charge as a switch in the **on** position, and a negative charge as a switch in the **off** position. Figure 7 shows the way that a computer scientist might think of a byte of memory: as a collection of switches that are each flipped to either the on or off position.

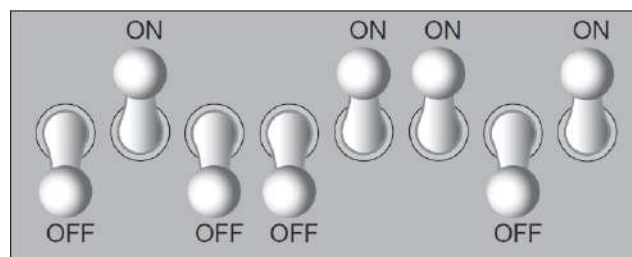


Figure 7 Think of a byte as eight switches (The number 77 stored in a byte).

When a piece of data is stored in a byte, the computer sets the eight bits to an on/off pattern that represents the data. For example, the pattern shown in Figure 7 shows how the number 77 would be stored in a byte, and the pattern in Figure 8 shows how the letter A would be stored in a byte.

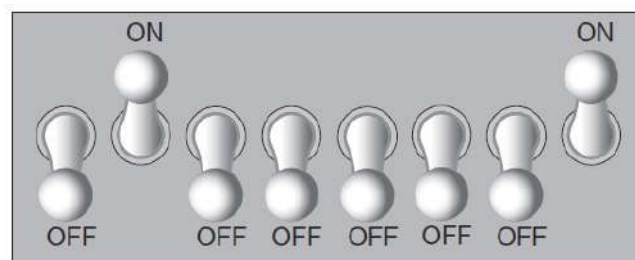


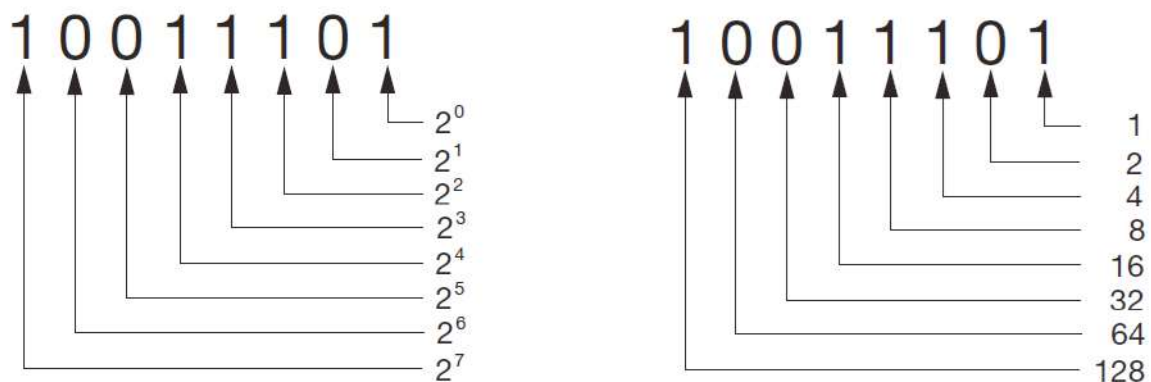
Figure 8 Think of a byte as eight switches (The letter A stored in a byte).

Storing Numbers

A bit can be used in a very limited way to represent numbers. Depending on whether the bit is turned on or off, it can represent one of two different values. In computer systems, a bit that is turned off represents the number **0** and a bit that is turned on represents the number **1**. This corresponds perfectly to the **binary numbering system**. In the binary numbering system (or **binary**, as it is usually called) all numeric values are written as sequences of 0s and 1s. Here is an example of a number that is written in binary:

10011101

The position of each digit in a binary number has a value assigned to it. Starting with the rightmost digit and moving left, the position values are 1, 2, 4, 8, and so forth, as shown in Figure 9-a. Figure 9-b shows the same diagram with the position values calculated. Starting with the rightmost digit and moving left, the position values are 1, 2, 4, 8, and so forth.



(a) The values of binary digits as powers of 2 (b) The values of binary digits

Figure 9

To determine the value of a binary number you simply add up the position values of all the 1s. For example, in the binary number 10011101, the position values of the 1s are 1, 4, 8, 16, and 128. This is shown in Figure 10. The sum of all of these position values is 157. So, the value of the binary number 10011101 is 157.

Figure 11 shows how you can picture the number 157 stored in a byte of memory. Each **1** is represented by a bit in the on position, and each **0** is represented by a bit in the off position.

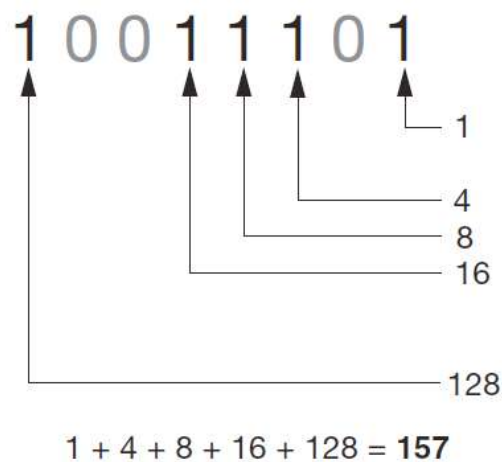


Figure 10 Determining the value of 10011101

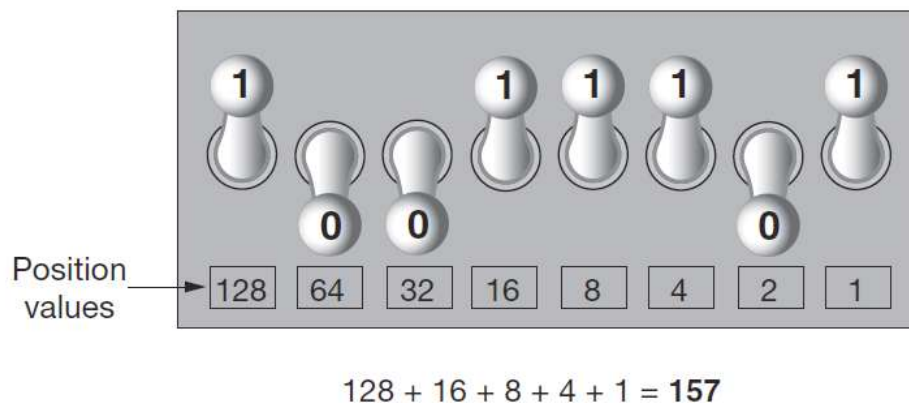


Figure 11 The bit pattern for 157

When all of the bits in a byte are set to 0 (turned off), then the value of the byte is 0. When all of the bits in a byte are set to 1 (turned on), then the byte holds the largest value that can be stored in it. The largest value that can be stored in a byte is $1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 = 255$. This limit exists because there are only eight bits in a byte.

What if you need to store a number larger than 255? The answer is simple: use more than one byte. For example, suppose we put two bytes together. That gives us 16 bits. The position values of those 16 bits would be 20, 21, 22, 23, and so forth, up through 215. As shown in Figure 12, the maximum value that can be stored in two bytes is 65,535. If you need to store a number larger than this, then more bytes are necessary.

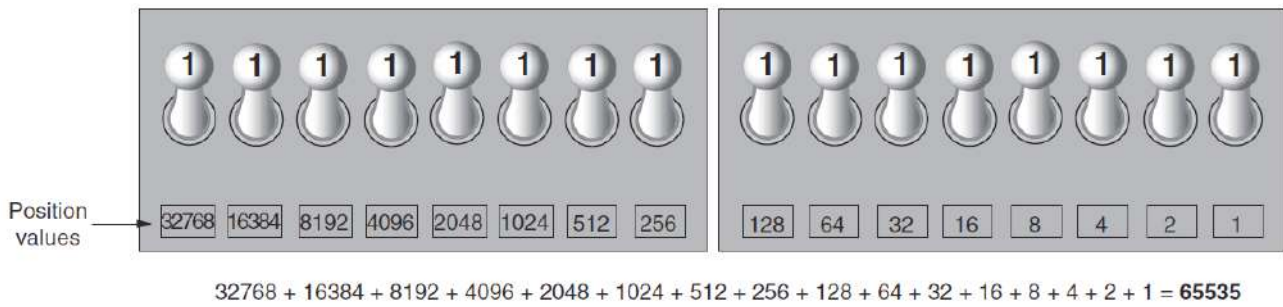


Figure 12 Two bytes used for a large number

Storing Characters

Any piece of data that is stored in a computer's memory must be stored as a binary number. That includes characters, such as letters and punctuation marks. When a character is stored in memory, it is first converted to a numeric code. The numeric code is then stored in memory as a binary number.

Over the years, different coding schemes have been developed to represent characters in computer memory. Historically, the most important of these coding schemes is **ASCII**, which stands for the **American Standard Code for Information Interchange**. ASCII is a set of 128 numeric codes that represent the English letters, various punctuation marks, and other characters. For example, the ASCII code for the uppercase letter A is 65. When you type an uppercase A on your computer keyboard, the number 65 is stored in memory (as a binary number, of course). This is shown in Figure 13.

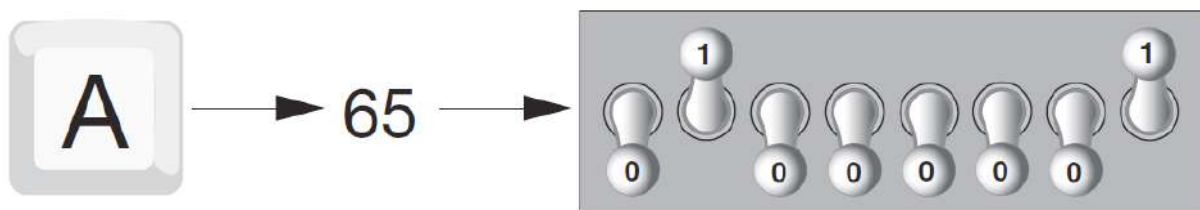


Figure 13 The letter A is stored in memory as the number 65

In case you are curious, the ASCII code for uppercase B is 66, for uppercase C is 67, and so forth. **Figure 14** shows all of the ASCII codes and the characters they represent.



TIP: The acronym ASCII is pronounced “askee.”

Code	Character	Code	Character	Code	Character	Code	Character	Code	Character
0	NUL	26	SUB	52	4	78	N	104	h
1	SOH	27	Escape	53	5	79	O	105	i
2	STX	28	FS	54	6	80	P	106	j
3	ETX	29	GS	55	7	81	Q	107	k
4	EOT	30	RS	56	8	82	R	108	l
5	ENQ	31	US	57	9	83	S	109	m
6	ACK	32	(Space)	58	:	84	T	110	n
7	BEL	33	!	59	;	85	U	111	o
8	Backspace	34	"	60	<	86	V	112	p
9	HTab	35	#	61	=	87	W	113	q
10	Line Feed	36	\$	62	>	88	X	114	r
11	VTAB	37	%	63	?	89	Y	115	s
12	Form Feed	38	&	64	@	90	Z	116	t
13	CR	39	'	65	A	91	[	117	u
14	SO	40	(	66	B	92	\	118	v
15	SI	41	)	67	C	93	]	119	w
16	DLE	42	*	68	D	94	^	120	x
17	DC1	43	+	69	E	95	_	121	y
18	DC2	44	,	70	F	96	`	122	z
19	DC3	45	-	71	G	97	a	123	{
20	DC4	46	.	72	H	98	b	124	
21	NAK	47	/	73	I	99	c	125	}
22	SYN	48	0	74	J	100	d	126	~
23	ETB	49	1	75	K	101	e	127	DEL
24	CAN	50	2	76	L	102	f		
25	EM	51	3	77	M	103	g		

Figure 14 ASCII Table (American Standard Code for Information Interchange)

The ASCII character set was developed in the early 1960s, and was eventually adopted by most all computer manufacturers. ASCII is limited, however, because it defines codes for only 128 characters. To remedy this, the Unicode character set was developed in the early 1990s. *Unicode* is an extensive encoding scheme that is compatible with ASCII, and can also represent the characters of many of the world's languages.

Advanced Number Storage

Negative numbers and real numbers (such as 3.14159) cannot be represented using the simple binary numbering technique we discussed earlier. Computers are able to store negative numbers and real numbers in memory, but to do so they use encoding schemes along with the binary numbering system. Negative numbers are encoded using a technique known as *two's complement*, and real numbers are encoded in *floating-point notation*. You don't need to know how these encoding schemes work, only that they are used to convert negative numbers and real numbers to binary format.

Lab 1: Introduction to C++ with a simple program implementation. Header files, Standard Input/output instructions, Comments in C++.

C++ Program Concepts

C++ program shown in Figure 15, explains what the various bits of it are:

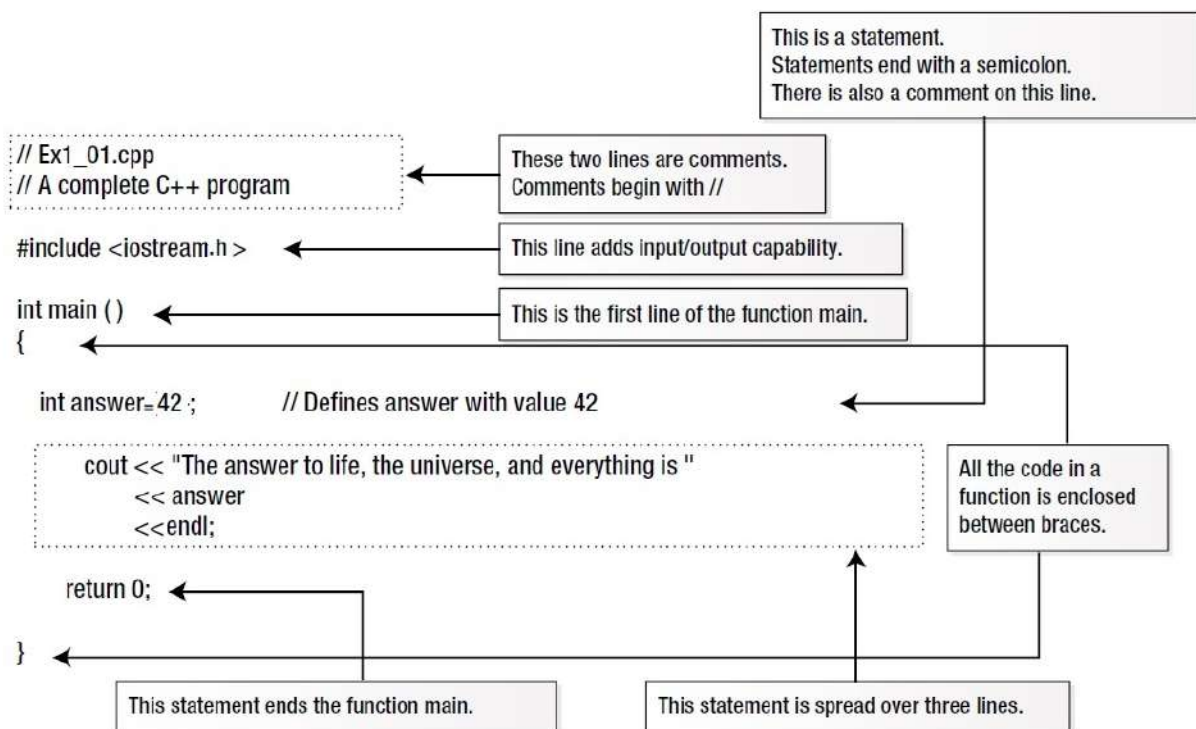


Figure 15 . A complete C++ program

The first two lines in Figure 15 are *comments*. You *add comments* that document your program code to make it easier for someone else to understand how it works. The compiler* ignores everything that follows two successive forward slashes on a line so this kind of comment can follow code on a line. The first line is a comment that indicates the name of the file containing this code. The file extension, `.cpp`, indicates that this is a C++ *source file*.

* A Compiler is a software that typically takes a high level language (Like C++ and Java) code as input and converts the input to a lower level language at once. It lists all the errors if the input code does not follow the rules of its language.

The program in Figure 15 consists of just the function `main()`. The first line of the function is: `int main()`

This is called the function header, which identifies the function. Here, `int` is a type name that defines the type of value that the `main()` function returns when it finishes execution - an integer. In general, the parentheses following a name in a function definition enclose the specification for information to be passed to the function when you call it.

A statement is a basic unit in a C++ program. A statement always ends with a semicolon and it's the semicolon that marks the end of a statement, not the end of the line. A statement defines something, such as a computation, or an action that is to be performed.

There are three statements in `main()` in Figure 15. The first defines a variable, which is a named bit of memory for storing data of some kind. In this case the variable has the name `answer` and can store integer values:

```
int answer = 42; // Defines answer with the value 42
```

The type, `int`, appears first, preceding the name. This specifies the kind of data that can be stored – integers. Note the space between `int` and `answer`. One or more whitespace characters is essential here to separate the type name from the variable name; without the space the compiler would see the name `intanswer`, which it would not understand. An initial value for `answer` appears after “=”, so it starts out storing 42.

There's a somewhat superfluous comment at the end of the first statement explaining what I just described but it does demonstrate that you can add a comments to a statement. The whitespace preceding the `//` is also not mandatory but it is desirable.

You can enclose several statements between a pair of curly braces, `{ }`, in which case they're referred to as a statement block. The body of a function is an example of a block, as you saw in Figure 15 where the statements in `main()` function appear between curly braces. A statement block is also referred to as a *compound statement* because in most circumstances it can be considered as a single statement, as you'll see later when we look at decision-making capabilities.

Input and output are performed using streams in C++. To output something, you write it to an output stream, and to input data you read it from an input stream. The standard output and input streams in C++ are called `cout` and `cin` respectively and by default they correspond to your computer's screen and keyboard.

The next statement in `main()` in Figure 15 outputs text to the screen:

```
cout << "The answer to life, the universe, and everything is "  
    << answer  
    << endl;
```

The statement is spread over three lines, just to show that it's possible. The names `cout` and `endl` are defined in the `iostream` header file. `<<` is the insertion operator that transfers data to a stream. Later you'll meet the extraction operator, `>>`, that reads data from a stream. Whatever appears to the right of each `<<` is transferred to `cout`. Writing `endl` to `cout` causes a new line to be written to the stream.

There's another form of comment that you can use when you need to spread a comment over several lines. For example:

```
/* This comment is  
over two lines. */
```

Everything between `/*` and `*/` will be ignored by the compiler. You can embellish this sort of comment to make it stand out. For example:

```
/******  
* This comment is *  
* over two lines. *  
******/
```

Whitespace is generally ignored by the compiler, except when it is necessary for syntactic reasons to distinguish one element from another.

The third line in Figure 15 is a preprocessing directive. Preprocessing directives cause the source code to be modified in some way before it is compiled to executable form. This preprocessing directive adds the contents of the standard library header file with the name `iostream` to this source file, `Ex1_01.cpp`. The header file contents are inserted in place of the `#include` directive.

Header files, which are sometimes referred to just as headers, contain definitions to be used in a source file. `iostream` contains definitions that are needed to perform input from the keyboard and text output to the screen using Standard Library routines.

The last statement in `main()` is a `return` statement. A `return` statement ends a function and returns control to where the function was called. In this case it ends the function and returns control to the operating system. A `return` statement may or may not return a value. This particular `return` statement returns 0 to the operating system. Returning 0 to the operating system indicates that the program ended normally. You can return non-zero values such as 1, 2, etc. to indicate different abnormal end conditions. The `return` statement in `Ex1_01.cpp` is optional, so you could omit it.

How a Program Works

A computer's CPU can only understand instructions that are written in machine language. Because people find it very difficult to write entire programs in machine language, other programming languages have been invented. Let us discuss this concept:

The CPU performs operations on pieces of data. The CPU does nothing on its own. It has to be told what to do, and that's the purpose of a program. A program is nothing more than a list of instructions that cause the CPU to perform operations.

Each instruction in a program is a command that tells the CPU to perform a specific operation. Here's an example of an instruction that might appear in a program:

10110000

To you and me, this is only a series of 0s and 1s. To a CPU, however, this is an instruction to perform an operation (*It is an actual instruction for an Intel microprocessor. It tells the microprocessor to move a value into the CPU*). It is written in 0s and 1s because CPUs only understand instructions that are written in machine language, and machine language instructions are always written in binary.

A machine language instruction exists for each operation that a CPU is capable of performing. For example, there is an instruction for adding numbers; there is an instruction for subtracting one number from another; and so forth. The entire set of instructions that a CPU can execute is known as the CPU's *instruction set*.

NOTE: There are several microprocessor companies today that manufacture CPUs. Some of the more well-known microprocessor companies are Intel, AMD, and Motorola. If you look carefully at your computer, you might find a tag showing a logo for its microprocessor. Each brand of microprocessor has its own unique instruction set, which is typically understood only by microprocessors of the same brand. For example, Intel microprocessors understand the same instructions, but they do not understand instructions for Motorola microprocessors.

The machine language instruction that was previously shown is an example of only one instruction. It takes a lot more than one instruction, however, for the computer to do anything meaningful. Because the operations that a CPU knows how to perform are so basic in nature, a meaningful task can be accomplished only if the CPU performs many operations. For example, if you want your computer to calculate the amount of interest that you will earn from your savings account this year, the CPU will have to perform a large number of instructions, carried out in the proper sequence. It is not unusual for a program to contain thousands, or even a million or more machine language instructions.

Programs are usually stored on a secondary storage device such as a disk drive. When you install a program on your computer, the program is typically copied to your computer's disk drive from a CD-ROM, Flash, or perhaps downloaded from a Web-site.

Although a program can be stored on a secondary storage device such as a disk drive, *it has to be copied into main memory, or RAM, each time the CPU executes it.* For example, suppose you have a word processing program on your computer's disk. To execute the program you use the mouse to double-click the program's icon. This causes the program to be copied from the disk into main memory. Then, the computer's CPU executes the copy of the program that is in main memory. This process is illustrated in Figure 16.

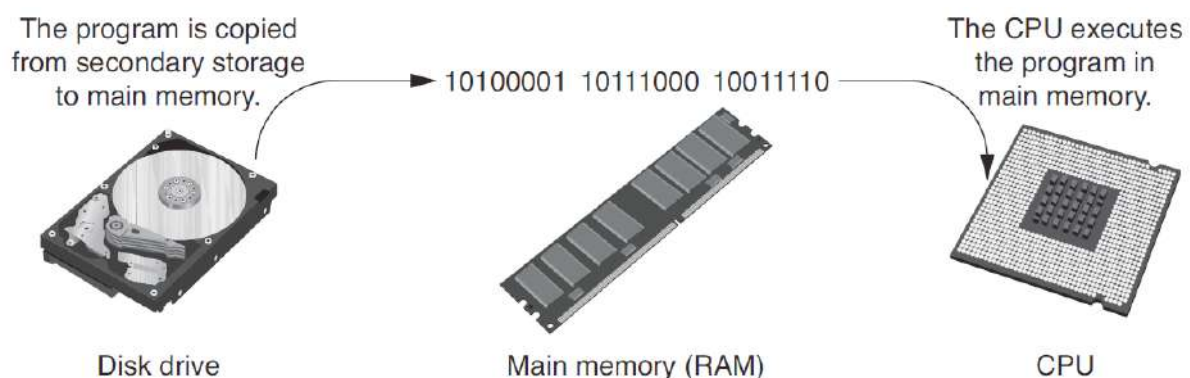


Figure 16: **A program is copied into main memory and then executed**

When a CPU executes the instructions in a program, it is engaged in a process that is known as the ***fetch-decode-execute cycle***. This cycle, which consists of three steps, is repeated for each instruction in the program. The steps are:

1. **Fetch** A program is a long sequence of machine language instructions. The first step of the cycle is to fetch, or read, the next instruction from memory into the CPU.

2. **Decode** A machine language instruction is a binary number that represents a command that tells the CPU to perform an operation. In this step the CPU decodes the instruction that was just fetched from memory, to determine which operation it should perform.

3. **Execute** The last step in the cycle is to execute, or perform, the operation.

Figure 17 illustrates these steps.

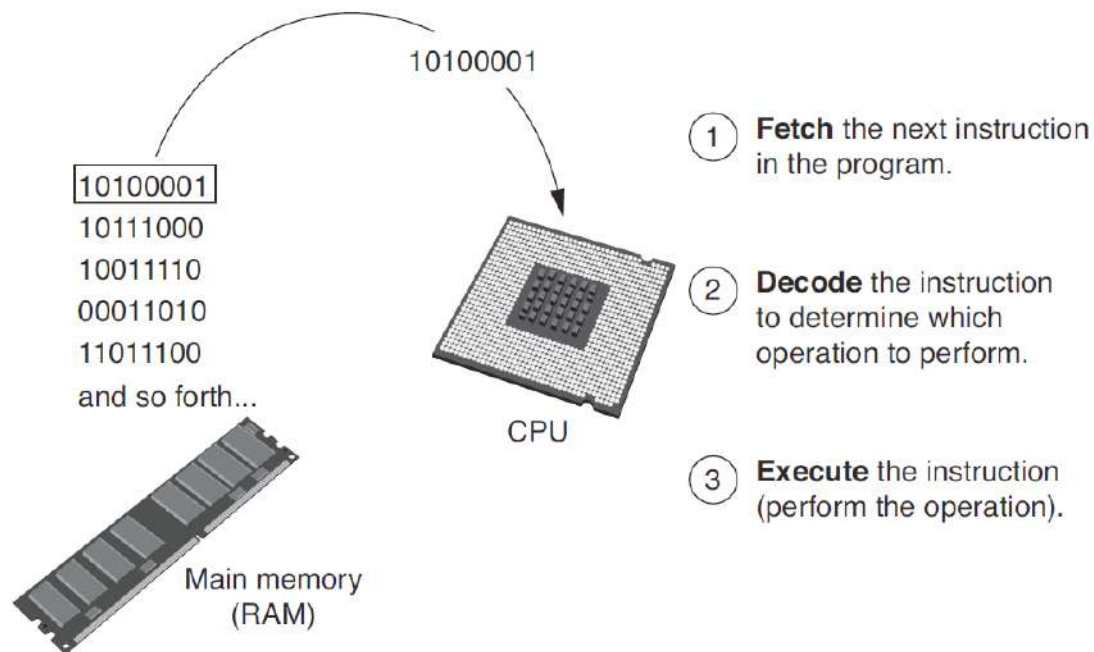


Figure 17: **The fetch-decode-execute cycle**

From Machine Language to Assembly Language

Computers can only execute programs that are written in machine language. As previously mentioned, a program can have thousands, or even a million or more binary instructions, and writing such a program would be very tedious and time consuming.

Programming in machine language would also be very difficult because putting a 0 or a 1 in the wrong place will cause an error.

Although a computer's CPU only understands machine language, it is impractical for people to write programs in machine language. For this reason, **assembly language** was created in the early days of computing as an alternative to machine language. Instead of using binary numbers for instructions, assembly language uses short words that are known as **mnemonics** (the initial "m" is silent). For example, in assembly language, the mnemonic

add typically means to add numbers, **mul** typically means to multiply numbers, and **mov** typically means to move a value to a location in memory. When a programmer uses assembly language to write a program, he or she can write short mnemonics instead of binary numbers.

NOTE: There are many different versions of assembly language. It was mentioned earlier that each brand of CPU has its own machine language instruction set. Each brand of CPU typically has its own assembly language as well.

Assembly language programs cannot be executed by the CPU, however. The CPU only understands machine language, so a special program known as an **assembler** is used to translate an assembly language program to a machine language program. This process is shown in Figure 18. The machine language program that is created by the assembler can then be executed by the CPU.

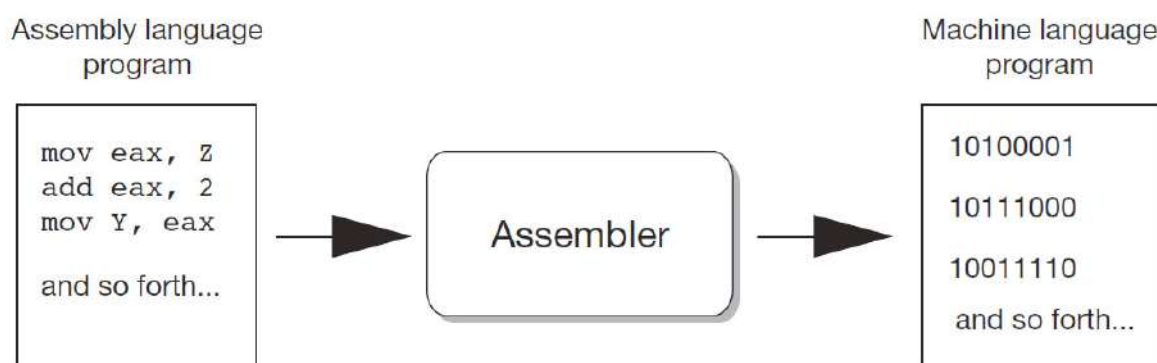


Figure 18: **An assembler translates an assembly language program to a machine language program**

High-Level Languages

Although assembly language makes it unnecessary to write binary machine language instructions, it is not without difficulties. Assembly language is primarily a direct substitute for machine language, and like machine language, it requires that you know a lot about the CPU. Assembly language also requires that you write a large number of instructions for even the simplest program. Because assembly language is so close in nature to machine language, it is referred to as a **low-level language**.

In the 1950s, a new generation of programming languages known as **high-level languages** began to appear. A high-level language allows you to create powerful and complex programs without knowing how the CPU works, and without writing large numbers of low-level instructions. In addition, most high-level languages use words that are easy to understand. For example, if a programmer were using COBOL (which was one of the early high-level languages created in the 1950s), he or she would write the following instruction to display the message “Hello world” on the computer screen:

Display "Hello world"

Doing the same thing in assembly language would require several instructions, and an intimate knowledge of how the CPU interacts with the computer’s video circuitry. As you can see from this example, high-level languages allow programmers to concentrate on the tasks they want to perform with their programs rather than the details of how the CPU will execute those programs. Since the 1950s, thousands of high-level languages have been created. Table 1 lists several of the more well-known languages.

Each high-level language has its own set of words that the programmer must learn in order to use the language. The words that make up a high-level programming language are known as **key words** or **reserved words**. Each key word has a specific meaning, and cannot be used for any other purpose. You previously saw an example of a COBOL statement that uses the key word *display* to print a message on the screen. In the Python language the word *print* serves the same purpose.

In addition to key words, programming languages have **operators** that perform various operations on data. For example, all programming languages have math operators that perform arithmetic. In Java, as well as most other languages, the + sign is an operator that adds two numbers. The following adds 12 and 75: **12 + 75**

In addition to key words and operators, each language also has its own **syntax**, which is a set of rules that must be strictly followed when writing a program. The syntax rules dictate how key words, operators, and various punctuation characters must be used in a program. When you are learning a programming language, you must learn the syntax rules for that particular language.

The individual instructions that you use to write a program in a high-level programming language are called **statements**. A programming statement can consist of key words,

operators, punctuation, and other allowable programming elements, arranged in the proper sequence to perform an operation.

Table 1 Programming languages

Language	Description
Ada	Ada was created in the 1970s, primarily for applications used by the U.S. Department of Defense. The language is named in honor of Countess Ada Lovelace, an influential and historical figure in the field of computing.
BASIC	Beginners All-purpose Symbolic Instruction Code is a general-purpose language that was originally designed in the early 1960s to be simple enough for beginners to learn. Today, there are many different versions of BASIC.
FORTRAN	FORmula TRANslator was the first high-level programming language. It was designed in the 1950s for performing complex mathematical calculations.
COBOL	Common Business-Oriented Language was created in the 1950s, and was designed for business applications.
Pascal	Pascal was created in 1970, and was originally designed for teaching programming. The language was named in honor of the mathematician, physicist, and philosopher Blaise Pascal.
C and C++	C and C++ (pronounced “c plus plus”) are powerful, general-purpose languages developed at Bell Laboratories. The C language was created in 1972 and the C++ language was created in 1983.
C#	Pronounced “c sharp.” This language was created by Microsoft around the year 2000 for developing applications based on the Microsoft .NET platform.
Java	Java was created by Sun Microsystems in the early 1990s. It can be used to develop programs that run on a single computer or over the Internet from a Web server.
JavaScript™	JavaScript, created in the 1990s, can be used in Web pages. Despite its name, JavaScript is not related to Java.
Python	Python is a general-purpose language created in the early 1990s. It has become popular in business and academic applications.
Ruby	Ruby is a general-purpose language that was created in the 1990s. It is increasingly becoming a popular language for programs that run on Web servers.
Visual Basic	Visual Basic (commonly known as VB) is a Microsoft programming language and software development environment that allows programmers to create Windows®-based applications quickly. VB was originally created in the early 1990s.

Compilers and Interpreters

Because the CPU understands only machine language instructions, programs that are written in a high-level language must be translated into machine language. Once a program has been written in a high-level language, the programmer will use a compiler or an interpreter to make the translation.

A **compiler** is a program that translates a high-level language program into a separate machine language program. The machine language program can then be executed any time it is needed. This is shown in Figure 19. As shown in the figure, compiling and executing are two different processes.

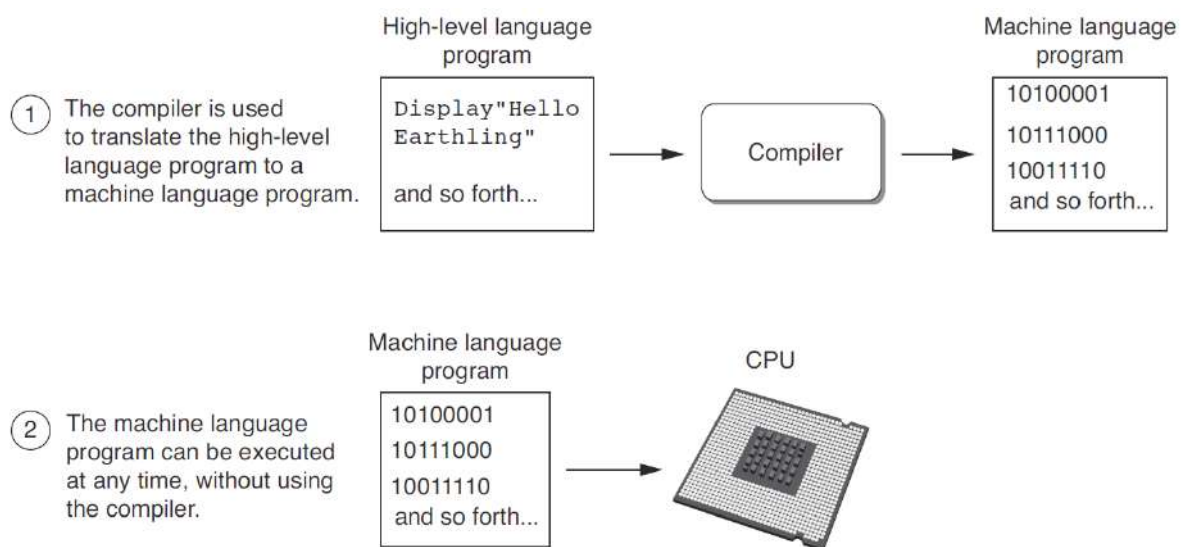


Figure 19: **Compiling a high-level program and executing it**

An **interpreter** is a program that both translates and executes the instructions in a high-level language program. As the interpreter reads each individual instruction in the program, it converts it to a machine language instruction and then immediately executes it. This process repeats for every instruction in the program. This process is illustrated in Figure 20. Because interpreters combine translation and execution, they typically do not create separate machine language programs.

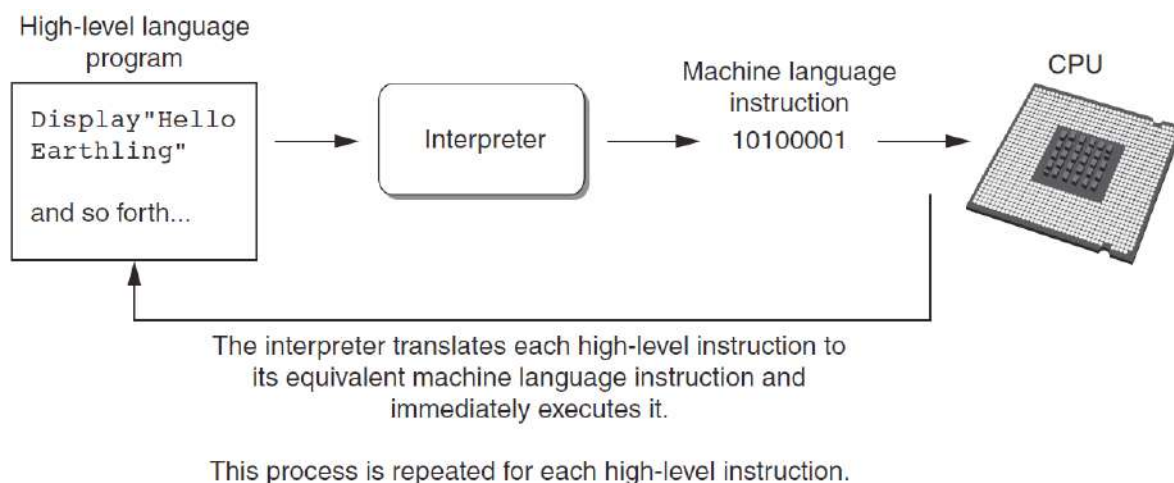


Figure 20: **Executing a high-level program with an interpreter**

NOTE: Programs that are compiled generally execute faster than programs that are interpreted because a compiled program is already translated entirely to machine language when it is executed. A program that is interpreted must be translated at the time it is executed.

The statements that a programmer writes in a high-level language are called **source code**, or simply *code*. Typically, the programmer types a program's code into a text editor and then saves the code in a file on the computer's disk. Next, the programmer uses a compiler to translate the code into a machine language program, or an interpreter to translate and execute the code. If the code contains a syntax error, however, it cannot be translated. A **syntax error** is a mistake such as a misspelled key word, a missing punctuation character, or the incorrect use of an operator. When this happens the compiler or interpreter displays an error message indicating that the program contains a syntax error. The programmer corrects the error and then attempts once again to translate the program.

Examples of pure compiled languages are C, C++. Examples of common interpreted languages are PHP, Ruby, Python, and JavaScript.

Integrated Development Environments

Although you can use a simple text editor such as Notepad (which is part of the Windows operating system) to write a program, most programmers use specialized software packages called **integrated development environments** or **IDEs**. Most IDEs combine the following programs into one software package: (A text editor that has specialized features for writing statements in a high-level programming language, a compiler or interpreter, and useful tools for testing programs and locating errors), see figure 21 for a C++ IDE.

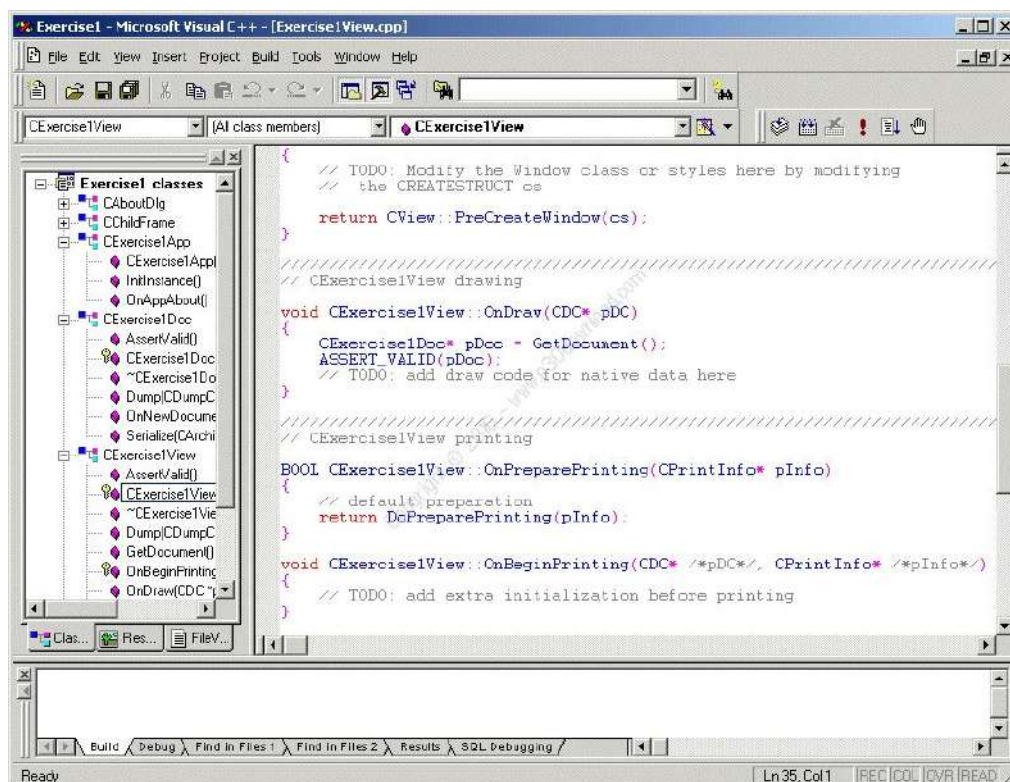


Figure 21: C++ IDE.

Designing a Program

The process of designing a program can be summarized in the following two steps:

1. Understand the task that the program is to perform.
2. Determine the steps that must be taken to perform the task.

Let's take a closer look at each of these steps.

Understand the Task That the Program Is to Perform

It is essential that you understand what a program is supposed to do before you can determine the steps that the program will perform. Typically, a professional programmer gains this understanding by working directly with the customer. We use the term **customer** to describe the person, group, or organization that is asking you to write a program. This could be a customer in the traditional sense of the word, meaning someone who is paying you to write a program. It could also be your boss, or the manager of a department within your company. Regardless of who it is, the customer will be relying on your program to perform an important task.

To get a sense of what a program is supposed to do, the programmer usually interviews the customer. During the interview, the customer will describe the task that the program should perform, and the programmer will ask questions to uncover as many details as possible about the task. A follow-up interview is usually needed because customers rarely mention everything they want during the initial meeting, and programmers often think of additional questions. The programmer studies the information that was gathered from the customer during the interviews and creates a list of different software requirements.

Determine the Steps That Must Be Taken to Perform the Task

Once you understand the task that the program will perform, you begin by breaking down the task into a series of steps. This is similar to the way you would break down a task into a series of steps that another person can follow. For example, suppose your little sister asks you how to boil water. Assuming she is old enough to be trusted around the stove, you might break down that task into a series of steps as follows:

1. Pour the desired amount of water into a pot.
2. Put the pot on a stove burner.
3. Turn the burner to high.
4. Watch the water until you see large bubbles rapidly rising. When this happens, the water is boiling.

This is an example of an **algorithm**, which is a set of well-defined logical steps that must be taken to perform a task. Notice that the steps in this algorithm are sequentially ordered.

Step 1 should be performed before Step 2, and so on. If your little sister follows these steps exactly as they appear, and in the correct order, she should be able to boil water successfully.

Algorithm: A sequence of steps used to solve a problem. The sequence presents a unique method of addressing an issue by providing a particular solution. Algorithm should be:

- *Finite*: The algorithm must eventually solve the problem.
 - *Well-defined*: The series of steps must be precise and understandable.
 - *Effective*: An algorithm must solve all cases of the problem.
-

A programmer breaks down the task that a program must perform in a similar way. An *algorithm* is created, which lists all of the logical steps that must be taken. For example, suppose you have been asked to write a program to calculate and display the gross pay for an hourly paid employee. Here are the steps that you would take:

1. Get the number of hours worked.
2. Get the hourly pay rate.
3. Multiply the number of hours worked by the hourly pay rate.
4. Display the result of the calculation that was performed in Step 3.

Of course, this algorithm isn't ready to be executed on the computer. The steps in this list have to be translated into code. Programmers commonly use two tools to help them accomplish this: *pseudocode* and *flowcharts*.

Pseudocode

The word *pseudo* means fake, so *pseudocode* (pronounced “sue doe code”) is fake code. It is an informal language that has no syntax rules, and is not meant to be compiled or executed. Instead, programmers use pseudocode to create models, or “mock-ups” of programs. Because programmers don't have to worry about syntax errors while writing pseudocode, they can focus all of their attention on the program's design. Once a satisfactory design has been created with pseudocode, the pseudocode can be translated directly to actual code.

Here is an example of how you might write pseudocode for *the pay calculating program* that we discussed earlier:

```
Display "Enter the number of hours the employee worked."  
Input hours  
Display "Enter the employee's hourly pay rate."  
Input payRate  
Set grossPay = hours * payRate  
Display "The employee's gross pay is $", grossPay
```

Each statement in the pseudocode represents an operation that can be performed in any high-level language.

Flowcharts

Flowcharting is another tool that programmers use to design programs. A **flowchart** is a diagram that graphically depicts the steps that take place in a program.

Figure 22 shows how you might create a flowchart for *the pay calculating program*. Notice that there are three types of symbols in the flowchart: ovals, parallelograms, and rectangles. The ovals, which appear at the top and bottom of the flowchart, are called *terminal symbols*. The *Start* terminal symbol marks the program's starting point and the *End* terminal symbol marks the program's ending point. Between the terminal symbols are parallelograms, which are used for both *input symbols* and *output symbols*, and rectangles, which are called *processing symbols*. Each of these symbols represents a step in the program. The symbols are connected by arrows that represent the "flow" of the program. To step through the symbols in the proper order, you begin at the *Start* terminal and follow the arrows until you reach the *End* terminal.

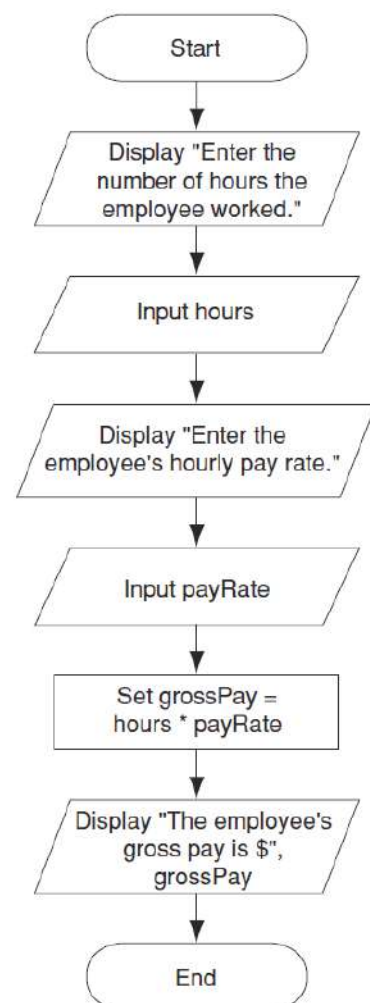


Figure 22: **Flowchart**

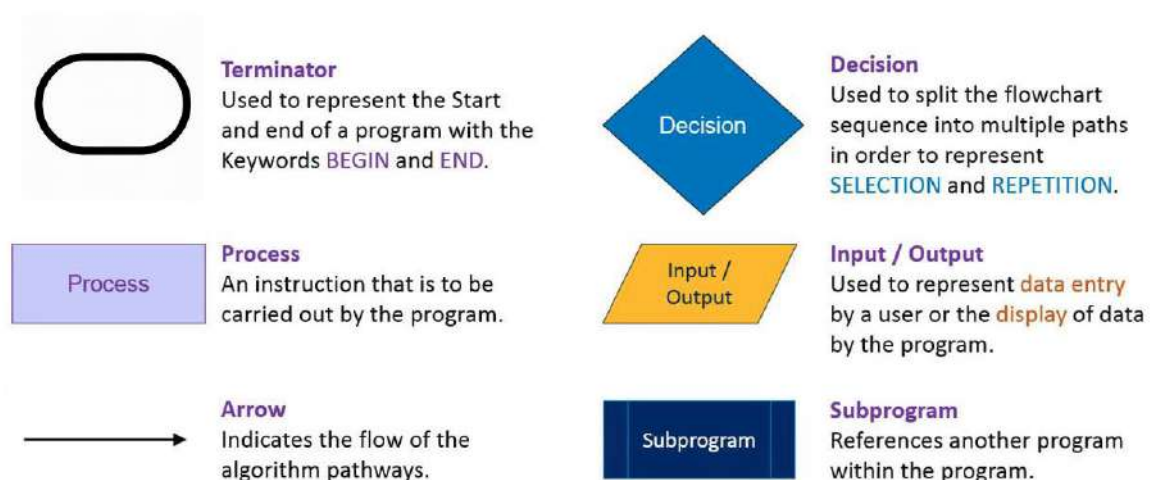
Characteristics of a good algorithm:

1. *Precision*: Each and every instruction should be precise and unambiguous i.e. each and every instruction should be clear and should have only one meaning
2. *Uniqueness*: Results of each step are uniquely defined and only depend on the input and the result of the preceding steps.
3. *Finiteness*: The algorithm stops after execution of a finite number of instructions
4. *Input*: The algorithm receives zero or more inputs.
5. *Output*: The algorithm produces output at least one output.
6. *Generality*: The algorithm applies to a set of inputs.

Advantages of algorithm:

1. It is a step-wise representation of a solution to a given problem, which makes it easy to understand.
2. An algorithm uses a definite procedure to solve a problem, meaning it has clear, unambiguous steps that can be followed to solve the problem.
3. It is not dependent on any programming language, so it is easy to understand for anyone even without programming knowledge.
4. Every step in an algorithm has its own logical sequence so it is easy to debug.
5. By using algorithm, the problem is broken down into smaller pieces or steps hence, it is easier for programmer to convert it into an actual program

Flowchart Symbols

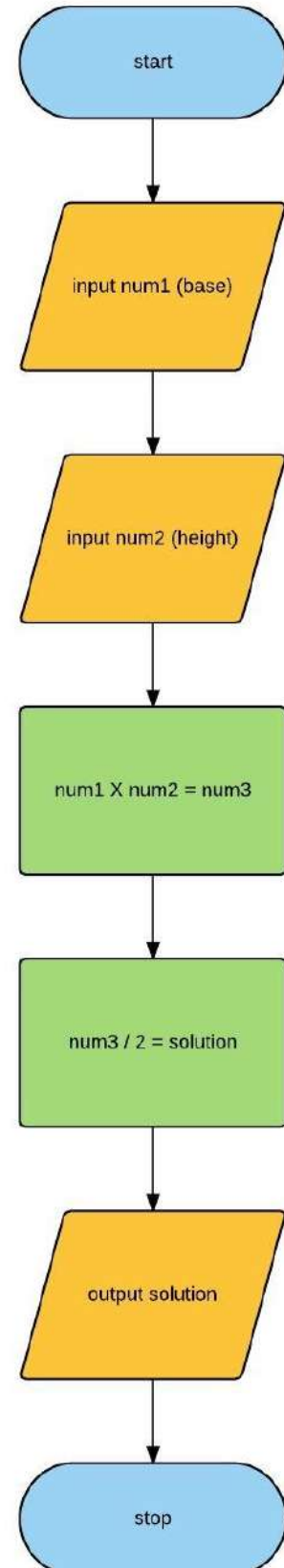


المحاضرة الثانية العملية:

1- Coding and executing a C++ program for the problem shown in the previous flowchart (Figure 22).

2- Coding and executing a C++ program for the following flowchart →

Can you explain what this algorithm is doing? →



Making Decisions

Decision-making is fundamental to any kind of computer programming. It's one of the things that differentiate a computer from a calculator. It means altering the sequence of execution depending on the result of a comparison.

Comparing Data Values

To make decisions, you need a mechanism for comparing things, and there are several kinds of comparisons. For instance, a decision such as, "If the traffic signal is red, stop the car," involves a comparison for equality. You compare the color of the signal with a reference color, red, and if they are equal, you stop the car. On the other hand, a decision such as, "If the speed of the car exceeds the limit, slow down," involves a different relationship. Here you check whether the speed of the car is greater than the current speed limit. Both of these comparisons are similar in that they result in one of two values: they are either true or false. This is precisely how comparisons work in C++.

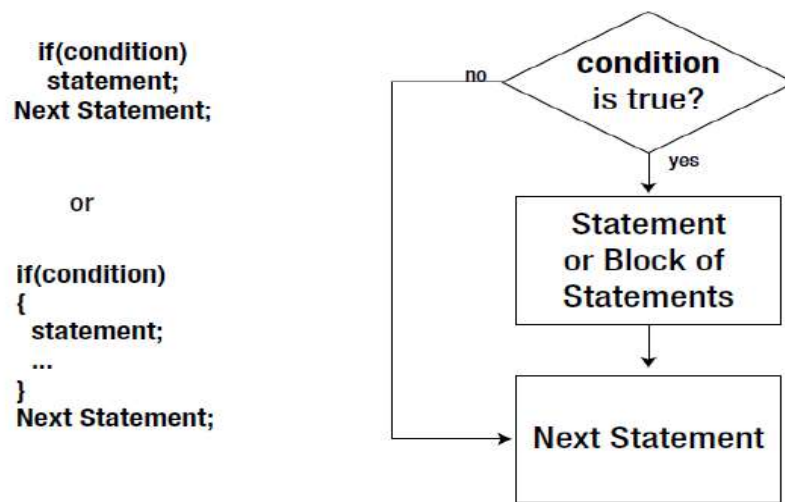
You can compare data values using some operators called relational operators. Table 2 lists the six operators for comparing two values.

Table 2 *Relational Operators*

Operator	Meaning
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
!=	not equal to

The if Statement

The basic `if` statement enables you to choose to execute a single statement, or a block of statements when a given condition is true. Figure 23 illustrates how this works.



The statement or block of statements that follows the **if** is only executed if **condition** is **true**.

Figure 23. Logic of the simple if statement

Here is an example of an **if** statement that tests the value of a char variable, letter:

```

if(letter == 'A')
    cout << "The first capital, alphabetically speaking.\n";
cout << "This statement always executes.\n";
  
```

If letter has the value 'A', the condition is true and these statements produce the following output:

```

The first capital, alphabetically speaking.
This statement always executes.
  
```

If the value of letter is not equal to 'A', only the second line appears in the output.

You put the condition to be tested between parentheses immediately following the keyword, **if**.

Notice the position of the semicolon (;). It goes after the statement following **if** and the condition between the parentheses. A semicolon (;) must not appear after the condition in parentheses, because **if** and the condition are bound with the statement or block that follows. They cannot exist by themselves.

Sometimes, you will see simple if statements written on a single line:

```

if(letter == 'A') cout << "The first capital, alphabetically speaking\n.";
  
```

You could extend code fragment to change the value of letter if it contains the value 'A':

```
if(letter == 'A')
{
    cout << "The first capital, alphabetically speaking.\n";
    letter = 'a';
}
cout << "This statement always executes.\n";
```

All the statements in the block will be executed when the **if** condition is true. Without the braces, only the first statement would be the subject of the **if**, and the statement assigning the value 'a' to letter would always be executed. Of course, each of the statements in the block are terminated by a semicolon but no semicolon is necessary after the closing brace of the block. You can have as many statements as you like within the block. Of course, the statement following the block always executes.

Nested if Statements

The statement that executes when the condition in an **if** statement is true can itself be an **if** statement. This arrangement is called a **nested if**. The condition of the inner if is only tested if the condition for the outer **if** is true. An **if** that is nested inside another can also contain a nested **if**. You can nest **ifs** to whatever depth you require.

```
#include <iostream.h>
int main()
{
    char letter;
    cout << "Enter a letter: "; // Prompt for the input
    cin >> letter;
    if(letter >= 'A') // Letter is 'A' or larger
        if(letter <= 'Z') // letter is 'Z' or smaller
            cout << "You entered an uppercase letter." << endl;
    if(letter >= 'a') // Test for 'a' or larger
        if(letter <= 'z') // letter is >= 'a' and <= 'z'
            cout << "You entered a lowercase letter." << endl;
    return 0;
}
```

After creating the `char` variable `letter`, the program prompts you to enter a letter. The `if` statement that follows checks whether the character entered is 'A' or larger. If `letter` is greater than or equal to 'A', the nested `if` that checks for the input being 'Z' or less executes. If it is 'Z' or less, you conclude that it is an uppercase letter and displays a message.

The next `if`, using essentially the same mechanism as the first, checks whether the character entered is lowercase, and displays a message.

You can use logical operators to combine a series of comparisons into a single expression so that you need just one `if`. There are three logical operators shown in Table 3.

Table 3 *Logical Operators*

Operator	Description
<code>&&</code>	Logical AND
<code> </code>	Logical OR
<code>!</code>	Logical negation (NOT)

Earlier, to determine whether a character was an uppercase letter, the value had to be both greater than or equal to 'A' and less than or equal to 'Z'. The `&&` operator only produces a true result if both operands are true. If either or both operands are false, then the result is false. Here's how you could test a `char` variable, `letter`, for an uppercase letter using the `&&` operator:

```
if(letter >= 'A' && letter <= 'Z')
{
    cout << "This is an uppercase letter." << endl;
}
```

The output statement executes only if both of the conditions combined by `&&` are true. No parentheses are necessary in the expression because the precedence of the comparison operators is higher than that of `&&`. As usual, you're free to put parentheses in if you want. You could write the statement as:

```
if((letter >= 'A') && (letter <= 'Z'))
{
    cout << "This is an uppercase letter." << endl;
}
```

Now there's no doubt that the comparisons will be evaluated first.

You can also use `isupper(letter)` or `islower(letter)` functions to test whether or not a character `letter` is an uppercase letter, by default 'A' to 'Z', or lowercase letter, by default 'a' to 'z'. In this case you must use the header `cctype.h`. Each of these functions returns a value of type `int`. The value will be non-zero (**true**) if the character is of the type being tested for, and **0 (false)** if it isn't. They also make the code simpler:

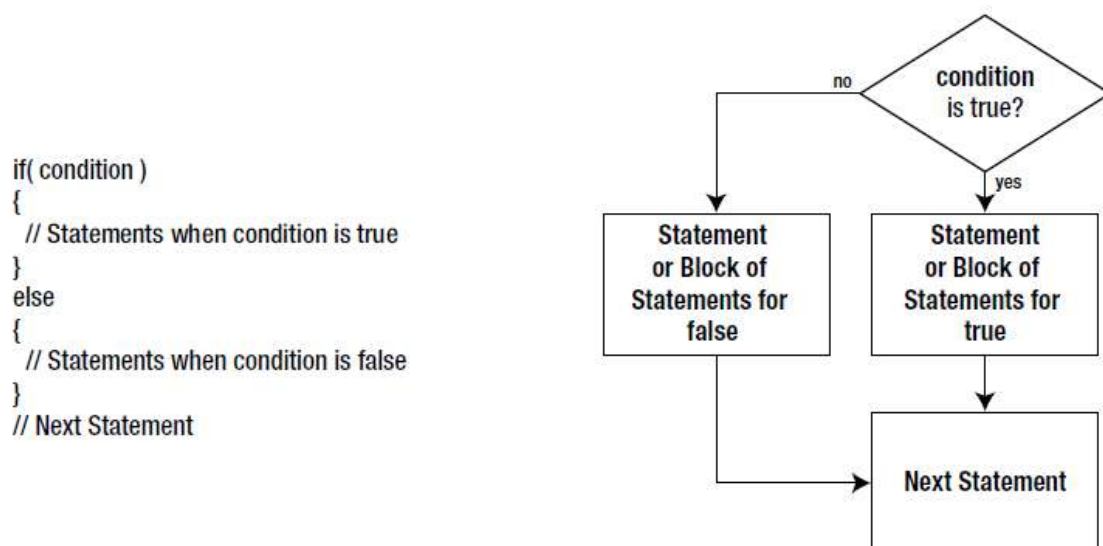
```
if(isupper(letter))
    cout << "You entered an uppercase letter." << endl;

if(islower(letter))
    cout << "You entered a lowercase letter." << endl;
```

The if-else Statement

The `if` statement that you have been using executes a statement or block if the condition specified is true. Program execution then continues with the next statement in sequence. Of course, you may want to execute one block of statements when the condition is true, and another set when the condition is false. An extension of the `if` statement called an **if-else** statement allows this.

The **if-else** combination provides a choice between two options. Figure 24 shows its general logic.



One of the two blocks in an if-else statement is always executed.

Figure 24. Logic of the if-else statement

You could write an **if-else** statement that would report whether the character stored in the **char** variable **character** was alphanumeric:

```
if(isalnum(character))
{
    cout << "It is a letter or a digit." << endl;
}
else
{
    cout << "It is neither a letter nor a digit." << endl;
}
```

This uses the **isalnum()** function from the locale header you saw earlier. If **character** contains a letter or a digit, **isalnum()** returns a positive integer. This will be implicitly converted to a **bool** value, which will be **true**, so the first message is displayed. If **character** contains other than a letter or a digit, **isalnum()** returns 0, which converts to **false** so the output statement after **else** executes. The braces are not mandatory here because they contain single statements but it's clearer if you put them in. The block following the **else** keyword is written without a semicolon appended, just like the **if** part of the statement.

Complete program:

```
#include <iostream.h>
#include <ctype.h>
void main()
{
    char character;
    cout<< "Enter a Char: ";
    cin>> character;
    if(isalnum(character))
    {
        cout << "It is a letter or a digit." << endl;
    }
    else
    {
        cout << "It is neither a letter nor a digit." << endl;
    }
}
```

Conditional Statements in C++

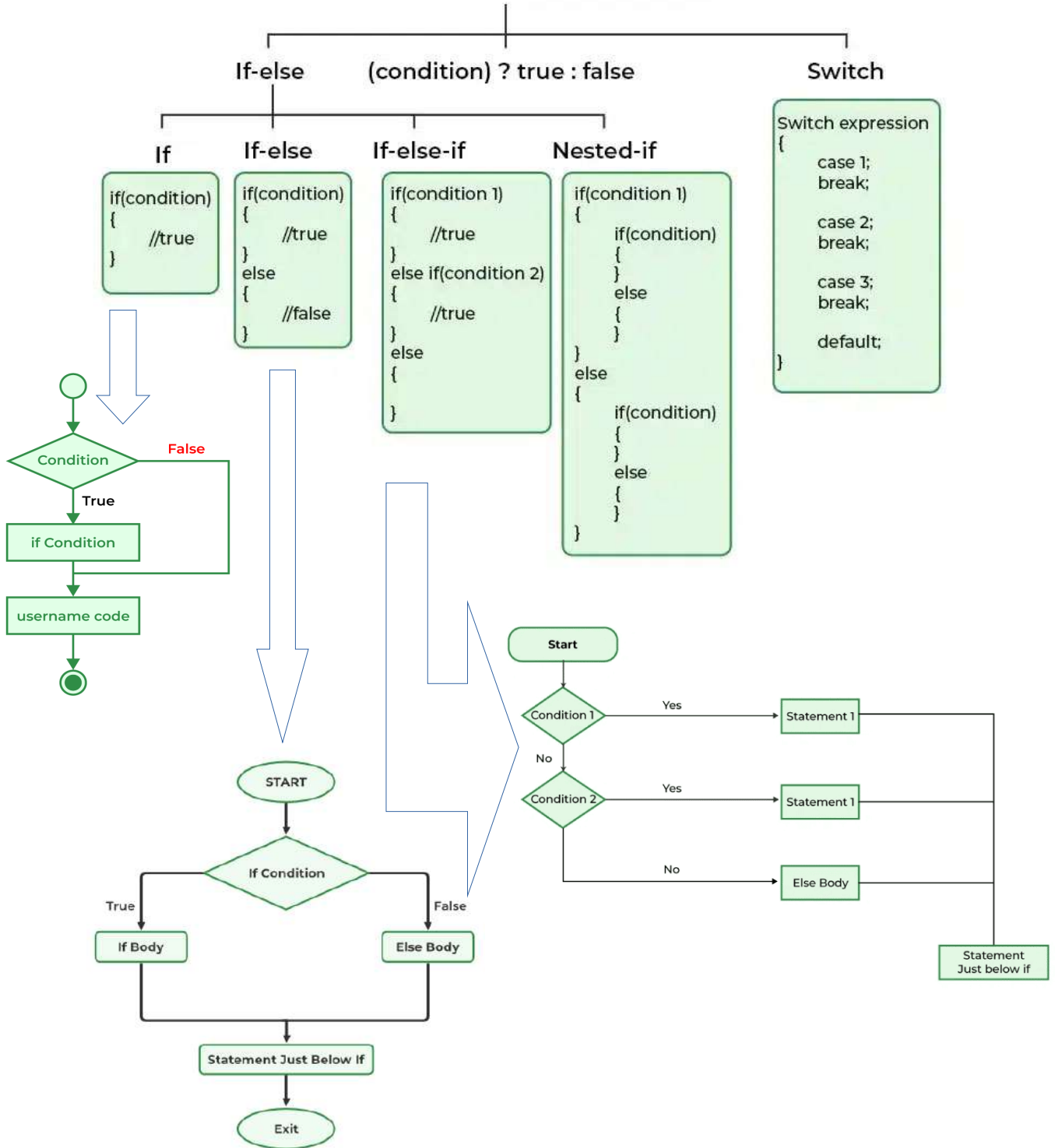


Figure 25. Types of Conditional Statements in C++

switch Statement in C++

The **switch case** statement is an alternative to the **if else if** ladder that can be used to execute the conditional code based on the value of the variable specified in the switch statement. The selection of a particular **case** is determined by the value of the integer expression between the parentheses that follow the keyword **switch**. The **default** label identifies the default case, which is selected if none of the other cases is selected. You don't have to specify a **default** case, though (If you don't, and none of the case values is selected, the **switch** does nothing). Figure 26 shows its general logic.

```
switch (expression)
{
    case value1:
        statements;
    case value2:
        statements;
    . . . .
    . . . .
    . . . .
    default:
        statements;
}
```

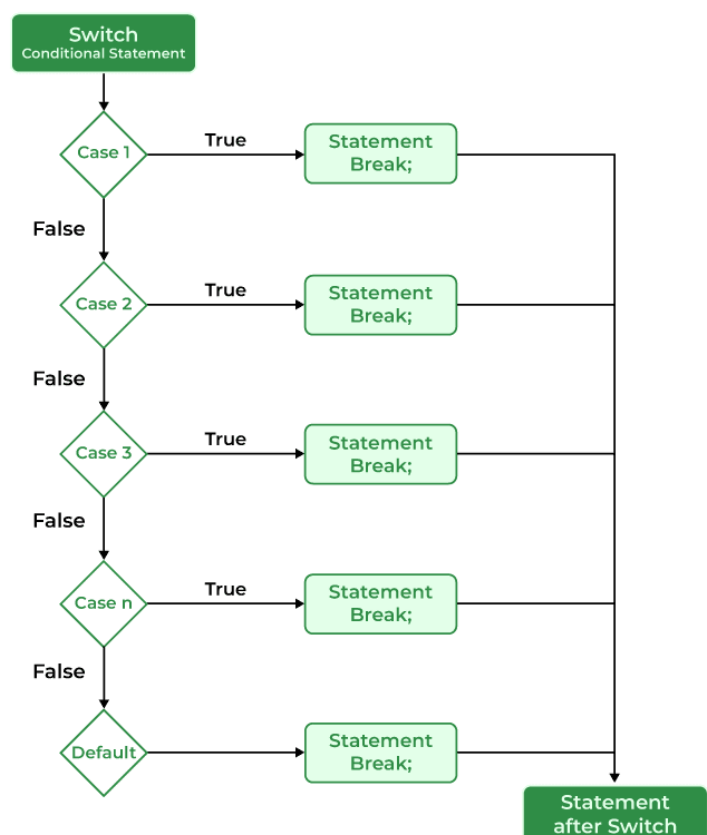


Figure 26. Logic of the switch Statement in C++

The **break** statement that appears after each set of case statements is essential for the logic here. Executing a *break* statement breaks out of the *switch* and causes execution to continue with the statement following the closing brace. If you omit the *break* statement for a case, the statements for the following *case* will execute. Notice that we don't need a *break* after the final case (usually the *default* case) because execution leaves the *switch* at this point anyway.

Note: The switch expression should evaluate to either integer or character. It cannot evaluate any other data type.

Example 1 of switch Statement:

```
// Using the switch statement
#include <iostream.h>
void main()
{
    int choice; // Stores selection value
    cout << "Your electronic recipe book is at your service.\n"
        << "You can choose from the following delicious dishes:\n"
        << "1 Boiled eggs\n"
        << "2 Fried eggs\n"
        << "3 Scrambled eggs\n"
        << "4 Coddled eggs\n\n"
        << "Enter your selection number: ";

    cin >> choice;

    switch(choice)
    {
        case 1:
            cout<< "\nBoil some eggs." << endl;
            break;
        case 2:
            cout << "Fry some eggs." << endl;
            break;
        case 3:
            cout << "Scramble some eggs." << endl;
            break;
        case 4:
            cout << "Coddle some eggs." << endl;
            break;
        default:
            cout << "You entered a wrong number - try raw eggs." << endl;
    }
}
```

Example 2 of switch Statement:

```
#include <iostream.h>
#include <ctype.h>
void main()
{
    char letter;
    cout << "Enter a letter: ";
    cin >> letter;
    if(isalpha(letter))
    {
        switch(tolower(letter))
        {
            case 'a': case 'e': case 'i': case 'o': case 'u':
                cout << "You entered a vowel." << endl;
                break;
            default:
                cout << "You entered a consonant." << endl;
        }
    }
    else
        cout << "You did not enter a letter." << endl;
}
```

Here is an example of some output:

Enter a letter: E

You entered a vowel.

The **if** condition first checks that you really do have a letter and not some other character using the **isalpha()** classification function. The integer returned will be nonzero if the argument is alphabetic and this will be implicitly converted to *true*, which causes the switch to be executed. The **switch** condition converts the value to lowercase using a Standard Library character conversion routine, **tolower()**, and uses the result to select a case. Converting to lowercase avoids the need to have case labels for upper and lowercase letters. All of the cases that identify a vowel cause the same statements to be executed. You can see

that you just write each of the cases in a series, followed by the statements any of the cases is to select. If the input is not a vowel, it must be a consonant and the **default** case deals with this.

If **isalpha()** returns 0, which converts to *false*, the **switch** doesn't execute because the else clause is selected; this outputs a message indicating that the character entered was not a letter.

The Conditional Operator

The conditional operator is sometimes called the **ternary operator** because it involves three operands — the only operator to do so. It parallels the if-else statement, in that instead of selecting one of two statement blocks to execute depending on condition, it selects the value of one of two expressions. Thus the conditional operator enables you to choose between two values. Let's consider an example: Suppose you have two variables, *a* and *b*, and you want to assign the value of the greater of the two to a third variable, *c*. The following statement will do this:

```
c = a > b ? a : b; // Set c to the higher of a and b
```

The conditional operator has a logical expression as its first operand, in this case **a > b**. If this expression is *true*, the second operand — in this case **a** — is selected as the value resulting from the operation. If the first operand is *false*, the third operand — in this case **b** — is selected as the value. The assignment statement is equivalent to the if statement:

```
if(a > b)  
  {  
    c = a;  
  }  
else  
  {  
    c = b;  
  }
```

المحاضرة الثالثة والرابعة العملية: تطبيق أمثلة برمجية عن اتخاذ القرار + امتحان عملي.

Mathematical Functions

The **math** header file defines a range of trigonometric and numerical functions that you can use in your programs. Table 4 presents some of the most useful functions from this header.

Table 4. *Some of the Numerical and Trigonometric Functions in the **math** header*

Function	Description
abs(arg)	Returns the absolute value of arg as the same type as arg , where arg can be of any floating-point type.
fabs(arg)	Returns the absolute value of arg as the same type as the argument. The argument can be int , long , float , double , or long double .
ceil(arg)	Returns a floating-point value of the same type as arg that is the smallest integer greater than or equal to arg , so ceil(2.5) produces 3.0 and ceil(-2.5) produces -2.0 . arg can be of any floating-point type.
floor(arg)	Returns a floating-point value of the same type as arg that is the largest integer less than or equal to arg so floor(2.5) results in 2.0 and floor(-2.5) results in -3.0 . arg can be any floating-point type.
exp(arg)	Returns the value of e^{arg} as the same type as arg . arg can be of any floating-point type.
log(arg)	Returns the natural logarithm (to base e) of arg as the same type as arg . arg can be any floating-point type.
log10(arg)	Returns the logarithm to base 10 of arg as the same type as arg . arg can be any floating-point type.
pow(arg1, arg2)	Returns the value of arg1 raised to the power arg2 , which is arg1^{arg2} . arg1 and arg2 can be integer or floating-point types. Thus the result of pow(2, 3) will be 8 , and the result of pow(1.5, 3) will be 3.375 .
cos(angle)	Returns the cosine of angle expressed in radians.
sin(angle)	Returns the sine of the angle expressed in radians.
tan(angle)	Returns the tangent of angle expressed in radians.
cosh(angle)	Returns the hyperbolic cosine of angle expressed in radians.
sinh(angle)	Returns the hyperbolic sine of angle expressed in radians.
tanh(angle)	Returns the hyperbolic tangent of angle expressed in radians.

Assignment No. 1: (Due date: 20/12)

1. Write a C++ program to find maximum between two numbers, and draw its flowchart.
2. Write a C++ program to find maximum between three numbers, and draw its flowchart.
3. Write a C++ program to check whether a number is negative, positive or zero, and draw its flowchart.
4. Write a C++ program to check whether a number is divisible by 5 or 7.
5. Write a C++ program to check whether a number is even or odd. and draw its flowchart.
6. Write a program to input a name of a person, his birth year. Then display his name, age and whether he was born in a leap year or not, and one of the following messages:
 "you are a child " if $13 > \text{age} \geq 10$.
 "you are a teenager " if $20 > \text{age} \geq 13$.
 "you are in adult " if $\text{age} \geq 20$.
 "you must be kidding !?" if $\text{age} < 1$, or , $\text{age} > 120$.
7. Write a C++ program to check whether a character is alphabet or not.
8. Write a C++ program to input any alphabet and check whether it is vowel or consonant.
9. Write a C++ program to input any character and check whether it is alphabet, digit, or special character.
10. Write a C++ program to check whether a character is uppercase or lowercase alphabet.
11. Write a C++ program to input week number and print week day.
12. Write a C++ program to input month number and print number of days in that month.
13. Write a C++ program to find all roots of a quadratic equation.
14. Write a C++ program to input marks of five subjects Physics, Chemistry, Biology, Mathematics and Programming. And find their average. Calculate final grade according to following:
 Average $\geq 90\%$: Grade A
 Average $\geq 80\%$: Grade B
 Average $\geq 70\%$: Grade C
 Average $\geq 60\%$: Grade D
 Average $\geq 50\%$: Grade E
 Average $< 50\%$: Grade F (Fail)
15. Write a C++ program to find x to the power of y , if y is less than 0 then use its absolute value.

Loops

A **loop** is a mechanism that enables you to execute a statement or block of statements repeatedly until a particular condition is met. The statements inside a loop are sometimes called *iteration statements*. A single execution of the statement or statement block that is within the loop is an iteration.

Two essential elements make up a loop: the statement or block of statements that forms the body of the loop that is to be executed repeatedly, and a *loop condition* of some kind that determines when to stop repeating the loop.

There are mainly two types of loops:

1.Entry Controlled loops: In this type of loop, the test condition is tested before entering the loop body. **for Loop** and **while Loop** is entry-controlled loops.

2.Exit Controlled Loops: In this type of loop the test condition is tested or evaluated at the end of the loop body. Therefore, the loop body will execute *at least once*, irrespective of whether the test condition is true or false. the **do-while loop** is exit controlled loop.

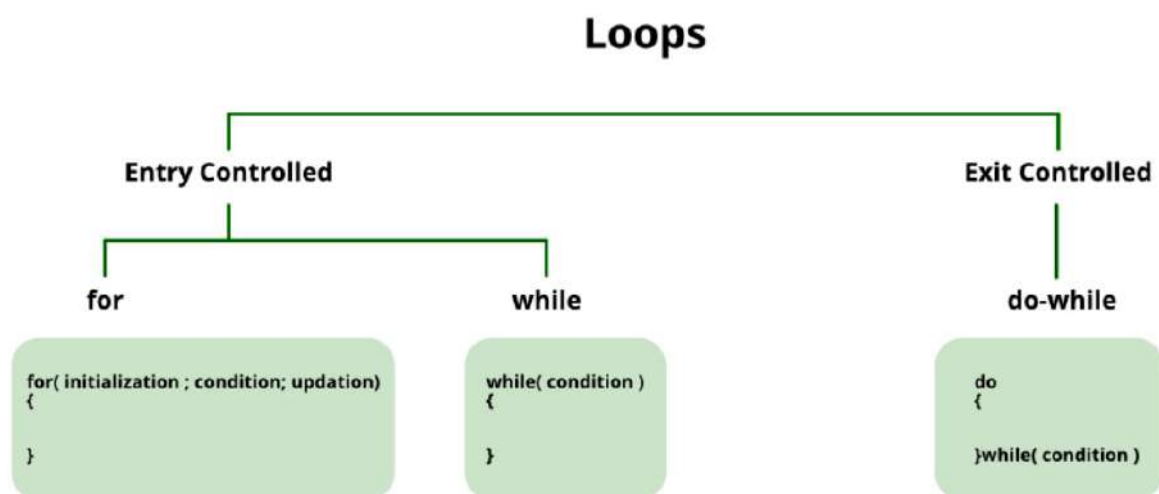


Figure 27. Types of loops

for loop – firstly initializes, then, condition check, execute body, update.

while loop – First checks the condition, then executes the body.

do-while loop – firstly, execute the body then condition check

The for Loop

The *for loop* executes a statement or block of statements a predetermined number of times. You specify how a for loop operates using three expressions separated by semicolons between parentheses following the *for* keyword. This is shown in Figure 28.

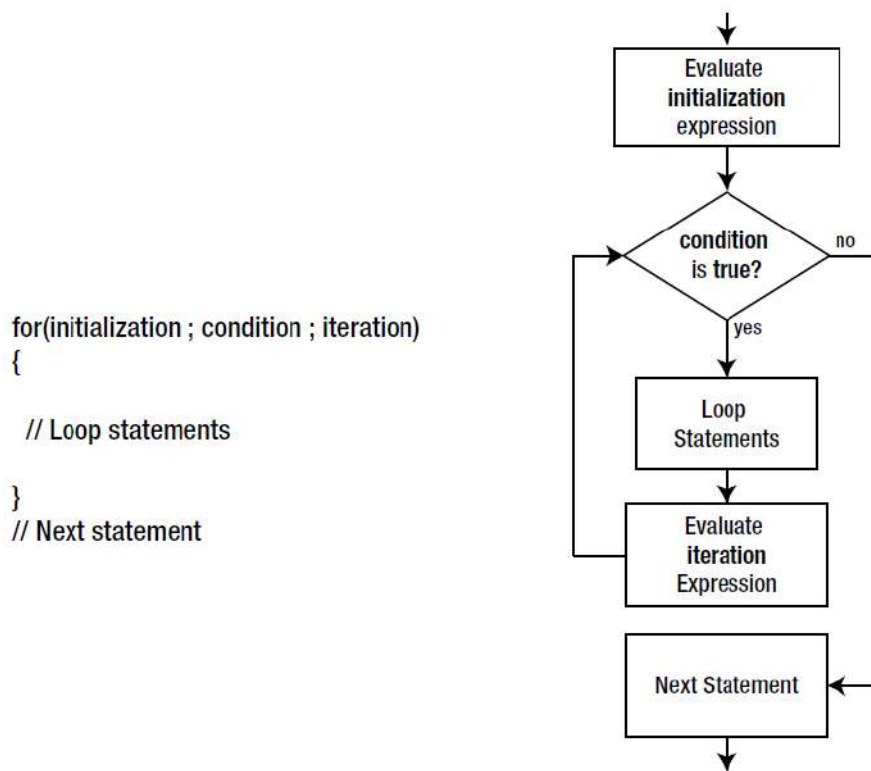


Figure 28. The logic of the for loop

The **initialization expression** is evaluated only once, at the beginning of the loop. The **loop condition** is checked next, and if it is true, the loop statement or statement block executes. If the condition is false, the loop ends and execution continues with the statement after the loop. After each execution of the loop statement or block, the **iteration expression** is evaluated and the condition is checked to decide if the loop should continue.

Example 1: A C++ program to find the sum and average of **N** given numbers.

```
#include <iostream.h>

void main()
{
    int N;
    cout << "How many numbers you want to find its average?\n";
    cin >> N;
```

```

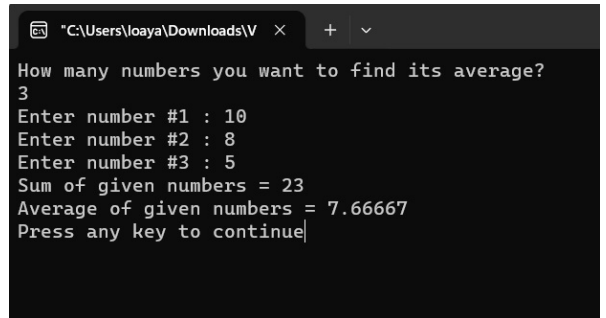
float sum = 0,a;

for (int I=0; I<=N-1; ++I)
{
cout<<"Enter number #"<< I+1<<" : ";
cin >> a;
sum =sum + a;
}

float average;

average = sum/N;
cout<< "Sum of given numbers = " << sum << endl;
cout<< "Average of given numbers = " << average << endl;
}

```



The screenshot shows a terminal window with the following text:

```

C:\Users\loaya\Downloads\V x + v
How many numbers you want to find its average?
3
Enter number #1 : 10
Enter number #2 : 8
Enter number #3 : 5
Sum of given numbers = 23
Average of given numbers = 7.66667
Press any key to continue

```

Example 2: Write a C++ program to find the partial sum for the following series:

$$x - \frac{x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} \dots \pm \frac{x^n}{n!}$$

```

#include <iostream.h>
#include <math.h>

void main()

{

cout <<" This program will find the partial sum of the following
series:"<<endl;

cout<< " x - x^2/2! + x^3/3! - x^4/4! ... upto Nth term"<<endl<<
endl;

int N;
double x;

cout << "How many number of terms in your series?\n";
cin >> N;

cout<< "Enter the value of x:\n";
cin>> x;

```

```

double sum = x, term = 1, fct, y=2, m;
int i, j;

for (i = 2; i <= N; i++)
{
    fct = 1;
    for (j = 1; j <= i; j++) {
        fct = fct * j;
    }
    term = term * (-1);
    m = term * pow(x, y) / fct;
    sum = sum + m;
    y++;
}

cout<< "partial sum of the series = " << sum << endl;
}

```

```

"C:\Users\loaya\Downloads\V"
This program will find the partial sum of the following series:
x - x^2/2! + x^3/3! - x^4/4! ... upto Nth term

How many number of terms in your series?
3
Enter the value of x:
2
partial sum of the series = 1.33333
Press any key to continue

```

Notes:

1- One can "count by" a value other than 1, as in this example:

```
for (count = 0; count < 100; count += 5)
```

2- You can follow the *for* statement with a null statement, allowing all the work to be done in the *for* statement itself. the null statement is a semicolon alone on a line. For example,

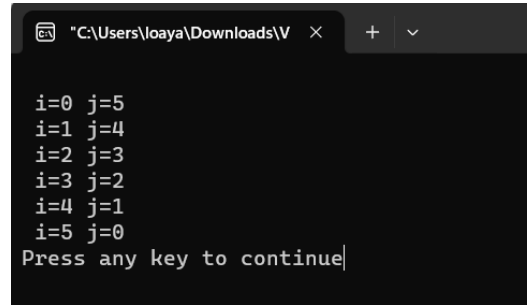
```
// print 1 to 10
```

```
for (count = 0; count < 10; cout<<"\n count = "<< ++count);
```

3- You can create an expression by separating two sub-expressions with the comma operator. By using the comma operator, you can make each part of a for statement perform multiple duties. For example,

//A program to display the number between 0 to 5, as well as 5 to 0 simultaneously one by one.

```
#include <iostream.h>
void main()
{
int i, j;
for (i = 0, j = 5; i <= 5; i++, j--)
    cout << "\n i=" << i << " j=" << j;
cout<< endl;
}
```



4- Sometimes, one may enter the for loop as:

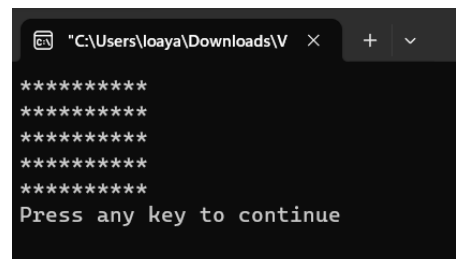
```
for ( ; ; )
```

Here, the **for** loop is valid but it will execute the loop indefinitely (forever!) because there is no condition to be checked to terminate this loop (it's a loop that never ends).

5- Nested for loops, any loop could be placed inside the other loop in C++ (as shown in example 2 before), here is another example:

//A program to display the '*', in 5 row of 10 columns each.

```
#include <iostream.h>
void main()
{
    int i , j, count=1;
    for (i = 1; i <= 5; cout<<endl, i++)
        for (j = 1; j <= 10; j++)
            cout<<'*';
}
```



المحاضرة الخامسة العملية: تطبيق أمثلة برمجية عن التكرار for loops + امتحان عملي.

The *while* Loop and the *do-while* loop

The ***while* loop** uses a logical expression to control execution of the loop body. The general form of the *while* loop is shown in Figure 29.

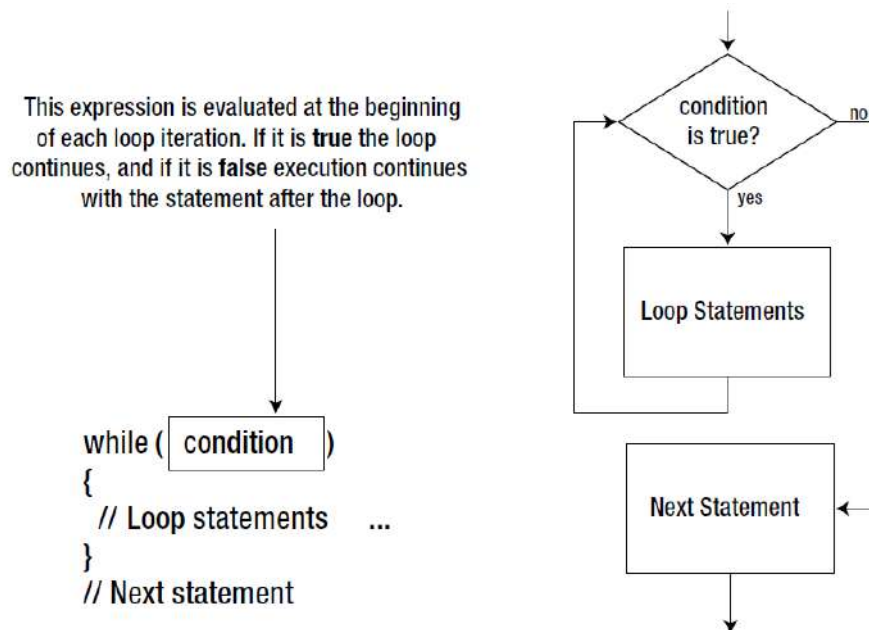


Figure 29. The logic of the *while* loop

The ***do-while* loop** is similar to the *while* loop in that the loop continues for as long as the specified loop condition remains true. However, the difference is that the loop condition is checked at the end of the *do-while* loop, rather than at the beginning, so the loop statement is always executed at least once. The logic and general form of the *do-while* loop are shown in Figure 30. Note that the semicolon that comes after the condition between the parentheses is absolutely necessary.

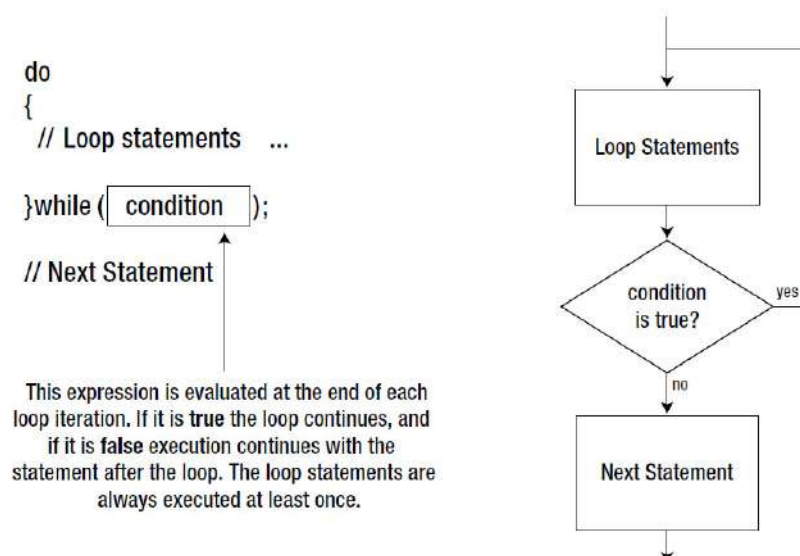


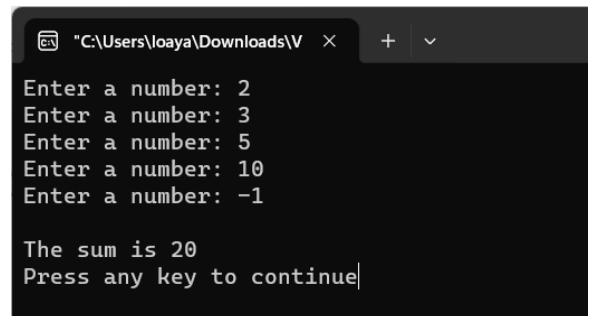
Figure 30. The logic of the *do-while* loop

Here is an example on using **while** loop:

```
// program to find the sum of positive numbers
// if the user enters a negative number, the loop ends
// the negative number entered is not added to the sum
#include <iostream.h>
int main() {
    int number;
    int sum = 0;
    // take input from the user for the first time
    cout << "Enter a number: ";
    cin >> number;

    while (number >= 0) {
        // add all positive numbers
        sum += number;

        // take input again if the number is positive
        cout << "Enter a number: ";
        cin >> number;
    }
    cout << "\nThe sum is " << sum << endl;    // display the sum
    return 0;
}
```

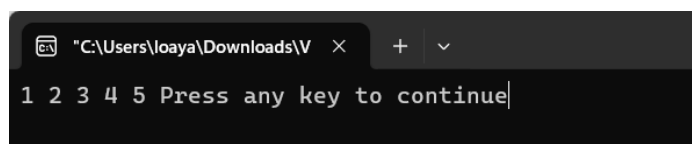


```
"C:\Users\loaya\Downloads\V x + v
Enter a number: 2
Enter a number: 3
Enter a number: 5
Enter a number: 10
Enter a number: -1

The sum is 20
Press any key to continue|
```

Here is an example on using **do -while** loop:

```
// C++ Program to print numbers from 1 to 5
#include <iostream.h>
int main() {
    int i = 1;
    do {
        // do...while loop from 1 to 5
        cout << i << " ";
        ++i;
    } while (i <= 5);
    return 0;
}
```



```
"C:\Users\loaya\Downloads\V x + v
1 2 3 4 5 Press any key to continue|
```

المحاضرة السادسة العملية: تطبيق أمثلة برمجية عن التكرار while & do-while.

Difference between `continue` and `break` statements in C++

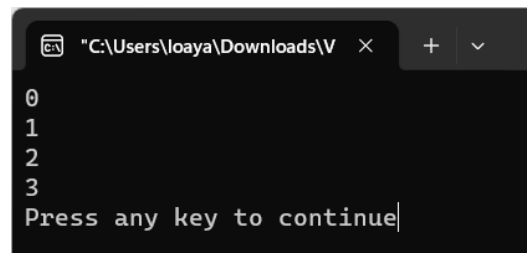
`Break` and **`continue`** are used to alter the normal flow of a program, you have already seen the **`break`** statement used in an earlier example with **`switch`** statement.

The **`break`** statement can also be used to jump out of a **loop**.

This example jumps out of the loop when **`i`** is equal to 4:

Example

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    cout << i << "\n";  
}
```



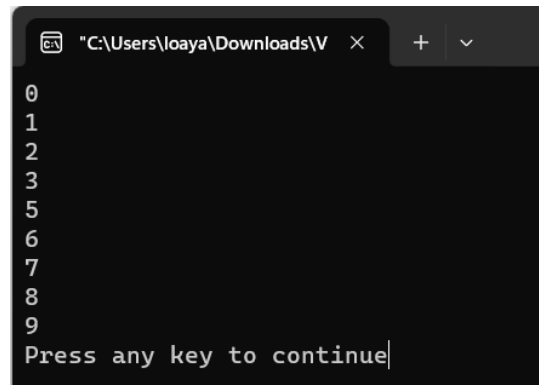
A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\loaya\Downloads\V". The output of the program is displayed as follows:
0
1
2
3
Press any key to continue|

The **`continue`** statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of 4:

Example

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    cout << i << "\n";  
}
```



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\loaya\Downloads\V". The output of the program is displayed as follows:
0
1
2
3
5
6
7
8
9
Press any key to continue|

(you can see that 4 is not displayed)

Using an Array

An **array** is a variable that represents a sequence of memory locations; each can store an item of data of the same data type.

For example, you could store 366 temperature samples in an array defined as follows:

```
double temperatures[366];    // An array of temperatures
```

This defines an array with the name **temperatures** to store 366 values of type **double**. The data values are called **elements**. The number of elements specified between the brackets is the **size** of the array. The array elements are not initialized in this statement so they contain junk values.

You refer to an array element using an integer called an **index**. The index of a particular array element is its offset from the first element. The first element has an offset of 0 and therefore an index of 0; an index value of 3 refers to the fourth array element — three elements from the first. To reference an element, you put its index between square brackets after the array name, so to set the fourth element of the **temperatures** array to 99.0, you would write:

```
temperatures[3] = 99.0;    // Set the fourth array element to 99
```

Let's look at another array. The Figure 31 shows the structure of an array called **height** that has six elements of type **double**.

height [0]	height [1]	height [2]	height [3]	height [4]	height [5]
26	37	47	55	62	75

Figure 31. An array with six elements

The array has six elements of type **int**. Each box in Figure 31 represents a memory location holding an array element. Each element can be referenced using the expression above it. You can define an array that has six elements of type **int** using this statement:

```
int height[6]; // Define an array of six heights
```

The compiler will allocate six contiguous storage locations for storing values of type **int** as a result of this definition. If this type is 4 bytes on your computer, this array will occupy 24

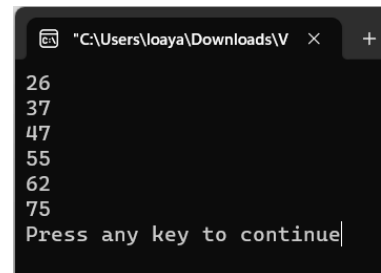
bytes. Each element in the height array in Figure 31 contains a different value. As there are six elements, the index values run from 0 for the first element through to 5 for the last element. You could define the array with these initial values like this:

```
// Define & initialize array of 6 heights
int height[6] = {26, 37, 47, 55, 62, 75};
```

The following program displays the elements of this array:

```
#include <iostream.h>

int main() {
    int height[6] = {26, 37, 47, 55, 62, 75};
    for (int i = 0; i < 6; i++) {
        cout << height [i] << "\n";
    }
    return 0;
}
```



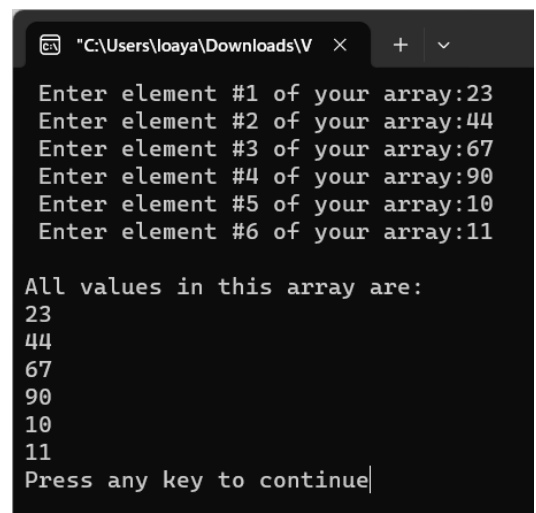
You can store different values in the array according to what the user wants to fill it as follows:

```
#include <iostream.h>

int main() {

    int height[6] , i;

    for (i = 0; i < 6; i++) {
        cout<<" Enter element #"<<i+1
            <<" of your array:";
        cin>> height [i];
    }
}
```



```
// Display your array
cout<< "\nAll values in this array are:"<<endl;
for (i = 0; i < 6; i++) {
    cout << height [i] << "\n";
}
return 0;
}
```

Multidimensional Arrays

All the arrays so far have required a single index value to select an element. Such an array is called a one-dimensional array, because varying one index can reference all the elements. You can also define arrays that require two or more index values to access an element. These are referred to generically as multidimensional arrays.

An array that requires two index values to reference an element is called a two-dimensional array. An array needing three index values is a three-dimensional array, and so on for as many dimensions as you think you can handle.

Suppose, as a gardener, that you want to record the weights of the carrots you grow in your small vegetable garden. To store the weight of each carrot, which you planted in three rows of four, you could define a two-dimensional array:

```
double carrots[3][4];
```

To reference a particular element of the carrots array, you need two index values. The first index specifies the row, from 0 to 2, and the second index specifies a particular carrot in that row, from 0 to 3. To store the weight of the third carrot (in the first row), you could write:

```
carrots[0][2] = 1.5;
```

Figure 32 illustrates this.

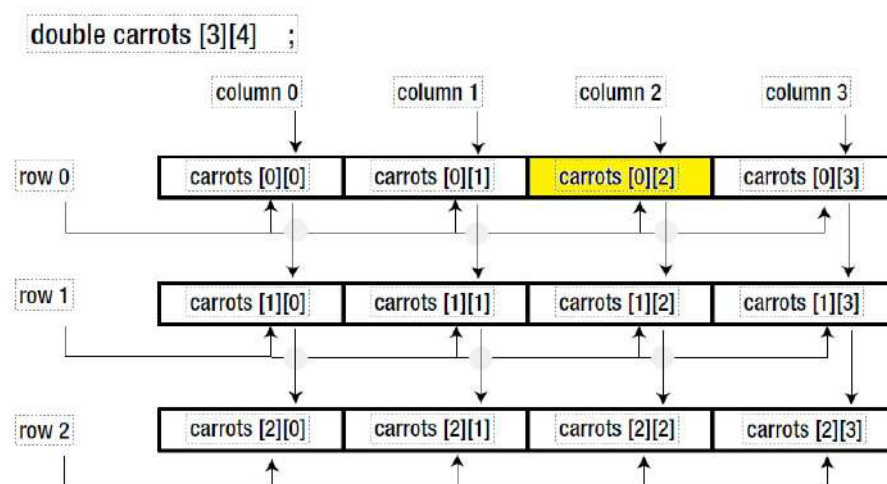


Figure 32. Rows and columns in a two-dimensional array

You could declare and initialize the two-dimensional carrots array, with this statement:

```
double carrots[3][4] = { {2.5, 3.2, 1.5, 4.1}, // First row
                          {4.1, 3.9, 1.6, 3.5}, // Second row
                          {2.8, 2.3, 0.9, 1.1}  // Third row
                        };
```

And you can read the weights of carrots into your multidimensional array and display it later as in the following program:

```
#include <iostream.h>

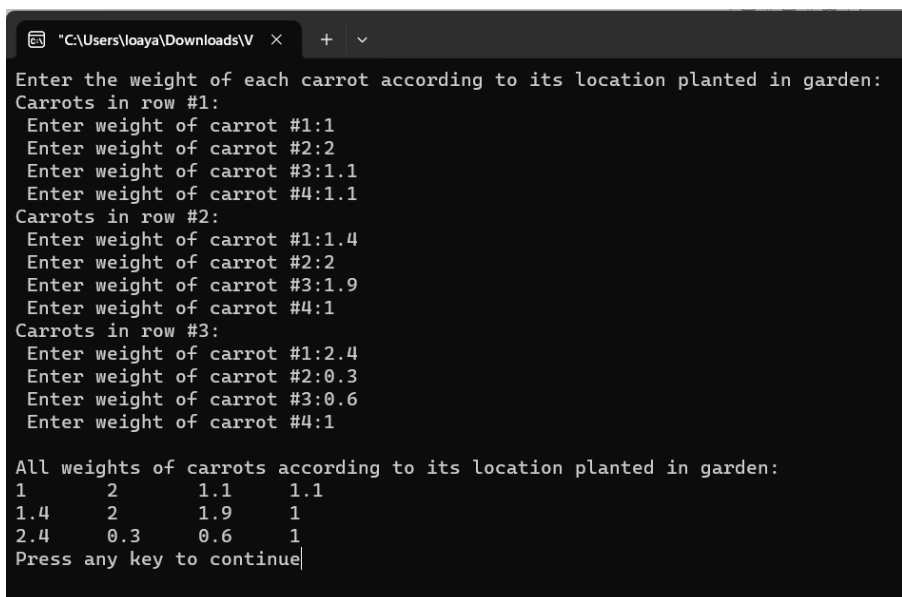
int main() {
    double carrots[3][4];
    int i, j;
    cout<< "Enter the weight of each carrot according to its location planted
in garden: \n";

    for (i = 0; i < 3; i++) {
        cout<<"Carrots in row #"<<i+1<<" : \n";
        for (j = 0; j < 4; j++) {
            cout<<" Enter weight of carrot #"<<j+1<<" : ";
            cin>> carrots[i][j];
        }
    }

    // Display your array
    cout<< "\nAll weights of carrots according to its location planted in
garden:"<<endl;
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            cout<< carrots[i][j]<< "\t";

        }
        cout<<endl;
    }

    return 0;
}
```



```
"C:\Users\loaya\Downloads\V" X + v
Enter the weight of each carrot according to its location planted in garden:
Carrots in row #1:
Enter weight of carrot #1:1
Enter weight of carrot #2:2
Enter weight of carrot #3:1.1
Enter weight of carrot #4:1.1
Carrots in row #2:
Enter weight of carrot #1:1.4
Enter weight of carrot #2:2
Enter weight of carrot #3:1.9
Enter weight of carrot #4:1
Carrots in row #3:
Enter weight of carrot #1:2.4
Enter weight of carrot #2:0.3
Enter weight of carrot #3:0.6
Enter weight of carrot #4:1
All weights of carrots according to its location planted in garden:
1      2      1.1    1.1
1.4    2      1.9    1
2.4    0.3    0.6    1
Press any key to continue
```

المحاضرة السابعة العملية: تطبيق أمثلة برمجية عن المصفوفات.

Files in C++

So far, we have been using the *iostream* standard library, which provides *cin* and *cout* methods for reading from standard input and writing to standard output respectively.

This tutorial will teach you how to read and write from a file. This requires another standard C++ library called *fstream*, which defines three new data types –

Sr.No	Data Type & Description
1	ofstream This data type represents the output file stream and is used to create files and to write information to files.
2	ifstream This data type represents the input file stream and is used to read information from files.
3	fstream This data type represents the file stream generally, and has the capabilities of both <i>ofstream</i> and <i>ifstream</i> which means it can create files, write information to files, and read information from files.

To perform file processing in C++, header files `<iostream.h>` and `<fstream.h>` must be included in your C++ source file.

Opening a File

A file must be opened before you can read from it or write to it. Either ***ofstream* or *fstream*** object may be used to **open a file for writing**. And ***ifstream*** object is used to **open a file for reading** purpose only.

Following is the standard syntax for *open()* function, which is a member of *fstream*, *ifstream*, and *ofstream* objects.

```
void open(const char *filename, ios::openmode mode);
```

Here, the first argument specifies the name and location of the file to be opened and the second argument of the *open()* member function defines the mode in which the file should be opened.

Let's look at the syntax of opening a file.

Like this:

```
#include <iostream.h>
#include <fstream.h>
int main()
{
    ofstream file;
    file.open ("d:/example.txt");
    file<<"Hello";
    return 0;
}
```

Or this:

```
#include <iostream.h>
#include <fstream.h>
int main(){
    { fstream file;

    file.open ("d:/example.txt", ios::out );

    file<<"Hello";

    return 0 ;

}
```

Note: if you use **fstream** instead of **ofstream** you must specify whether you want to open the file for read or write or both "read and write".

We have opened the file 'example.txt' to write on it. The file is located in **partition D** of your hard disk, you can specify a different path as you like.

We can also open the file for both reading and writing purposes. Let's see how to do this:

```
#include <iostream.h>
#include <fstream.h>
int main(){
    fstream file;

    file.open ("d:/example.txt", ios::out | ios::in );

    file<<"Hello";

    file.seekg(0, ios::beg); // Return to beginning of file

    char c;

    while (!file.eof() )
    {
        file.get(c);

        cout << c;

    }

    return 0;
}
```

Sr.No	Mode Flag & Description
1	ios::app Append mode. All output to that file to be appended to the end.
2	ios::ate Open a file for output and move the read/write control to the end of the file.
3	ios::in Open a file for reading.
4	ios::out Open a file for writing.
5	ios::trunc If the file already exists, its contents will be truncated before opening the file.

Another example for opening a file to read and count the number of lines in a file. The file must contain text previously otherwise there will be zero lines.

```
#include <iostream.h>
#include <fstream.h>

int main() {
    ifstream fin;
    fin.open("d:/my-input-file.txt", ios::in);

    char my_character ;
    int number_of_lines = 0;

    while (!fin.eof() ) {

        fin.get(my_character);
        cout << my_character;
        if (my_character == '\n' || fin.eof()){
            ++number_of_lines;
        }
    }
    cout << "NUMBER OF LINES: " << number_of_lines << endl;

    Return 0;
}
```

Closing a File

C++ automatically close and release all the allocated memory. But a programmer should always close all the opened files. Let's see how to close it.

```
#include <iostream.h>
#include <fstream.h>
int main(){
    ofstream file;
    file.open ("d:/example.txt");
    file.close();
    return 0;
```

Writing to a File

While doing C++ programming, you write information to a file from your program using the stream insertion operator (<<) just as you use that operator to output information to the screen. The only difference is that you use an *ofstream* or *fstream* object instead of the *cout* object.

Reading from a File

You read information from a file into your program using the stream extraction operator (>>) just as you use that operator to input information from the keyboard. The only difference is that you use an *ifstream* or *fstream* object instead of the *cin* object.

Read and Write Example

Following is a C++ program which opens a file in reading and writing mode. After writing information entered by the user to a file named *afile.dat*, the program reads information from the file and outputs it onto the screen:

```

#include <fstream.h>
#include <iostream.h>

int main () {
    char data[100] , c;
    int age;
    // open a file in write mode.
    ofstream outfile;
    outfile.open("d:/ afile.txt");

    cout << "Writing to the file" << endl;
    cout << "Enter your name: ";
    cin.getline(data, 100);

    // write inputted data into the file.
    outfile << data << endl;

    cout << "Enter your age: ";
    cin >> age;

    // again write inputted data into the file.
    outfile << age << endl;

    // close the opened file.
    outfile.close();
}

```

```
// open a file in read mode.
ifstream infile;
infile.open("d:/ afile.txt");

cout << "Reading from the file" << endl;

// write the data at the screen.
while (!infile.eof() )
{
    infile.get(c);
    cout << c;
}

// close the opened file.
infile.close();

return 0;
}
```

Output

```
Writing to the file
Enter your name: Ahmed Ali Adnan
Enter your age: 30
Reading from the file
Ahmed Ali Adnan
30
```

Functions in C++

A **function** is a set of statements that when called perform some specific task. It is the basic building block of a C++ program that provides modularity and code reusability. The programming statements of a function are enclosed within **{ }** braces, having certain meanings and performing certain operations, they are self-contained units. They are also called subroutines or procedures in other languages. The statements in function bodies are written only once but are reused from perhaps several locations in a program. A function should be designed so that it has one primary task to accomplish.

The main advantages of using functions are:

1. Easy to write a correct small function.
2. Easy to read, write and debug a function.
3. Easier to maintain or modify such a function.
4. Small functions tend to be self-documenting and highly readable.
5. It can be called any number of times in any place with different parameters.

Library Functions

These functions are the built-in functions i.e., they are predefined in the library of the C++. These are used to perform the most common operations like calculations. Some of the library functions are `sqrt()`, `abs()`, `tan()`, etc. To use these functions in the program the user has to use a header file associated with the corresponding function in the program.

User-defined Functions

These functions are designed by the user (it means the programmer will write it!) when they are writing any program because for every task we do not have a library of functions where their definitions are predefined. To perform according to the requirement of the user, the user has to develop some functions by itself, these functions are called user-defined functions. For such functions, the users can write their own logic according to the requirement. The general format of the function is given below:

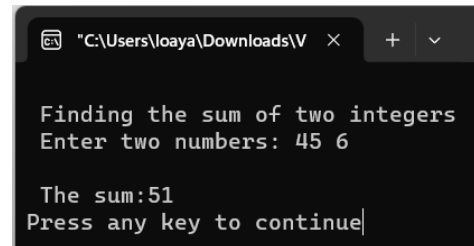
```
function_type function_name (datatype argument1, datatype argument2, ... )  
  
    {  
        //body of function  
        //process on the arguments  
        return something;  
    }
```

Example 1: A C++ program with a function to add two integer numbers.

```
#include <iostream.h>

int AddTwoNumbers(int n , int m)
{
    int total;
    total = n + m;
    return total;
}

int main()
{
    int x,y, sum;
    cout <<"\n Finding the sum of two integers ";
    cout <<"\n Enter two numbers: ";
    cin >>x>>y;
    sum = AddTwoNumbers(x,y);
    cout << "\n The sum:" << sum << endl;
    return 0;
}
```



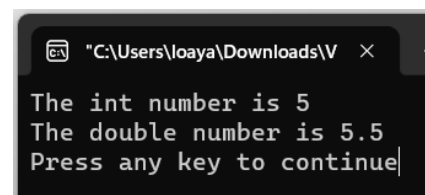
A screenshot of a terminal window showing the execution of the C++ program. The window title is "C:\Users\loaya\Downloads\V". The output text is: "Finding the sum of two integers", "Enter two numbers: 45 6", "The sum:51", and "Press any key to continue".

Example 2: A C++ program with a function to display two numbers.

```
#include <iostream.h>

// displaying two numbers passed to the function through arguments
void displayNum(int n1, float n2)
{
    cout << "The int number is " << n1;
    cout << "The double number is " << n2;
}

int main()
{
    int num1 = 5;
    double num2 = 5.5;
    displayNum(num1, num2); // calling the function
    return 0;
}
```



A screenshot of a terminal window showing the execution of the C++ program. The window title is "C:\Users\loaya\Downloads\V". The output text is: "The int number is 5", "The double number is 5.5", and "Press any key to continue".

If we want to define a function after the function call, we need to use the function prototype.

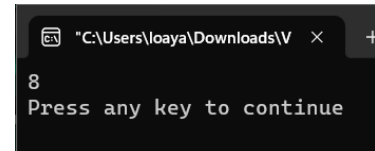
See Example 3:

```
#include <iostream.h>

void add(int, int); // function prototype, or you can say function declaration

int main()
{
    add(5, 3); // calling the function
    return 0;
}

void add(int a, int b) // function definition, void means it doesn't
{                       // return a value to the main function
    cout << (a + b) << endl;
}
```



Example 4: A C++ program to find the factorial of a positive integer using a function.

Note: The Factorial of a non-negative integer n is the multiplication of all integers smaller than or equal to n . For example, if $n=6$ then the factorial of 6 is $6*5*4*3*2*1$ which is 720.

```
#include <iostream.h>

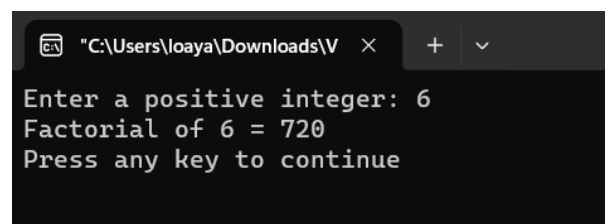
int factorial(int);

int main() {
    int n, f;

    cout << "Enter a positive integer: ";
    cin >> n;

    if (n < 0)
        cout << "Error! Factorial of a negative number doesn't exist.";
    else {
        f = factorial(n);
        cout << "Factorial of " << n << " = " << f << endl;
    }
    return 0;
}

int factorial (int x)
{
    int mul = 1;
    for(int i = 1; i <= x; ++i) {
        mul *= i;
    }
    return mul;
}
```

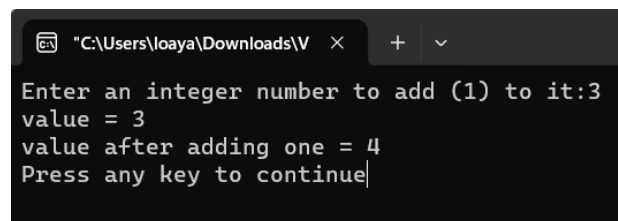


Several ways exist in which data (or variables) could be sent as an argument to a function. Two of the common ones are Passing by Value and Passing by Reference. In example 1, 2, 3 and 4 parameters were passed by value to the function.

Passing by reference allows a function to modify a variable without creating a copy. We have to declare reference variables. The memory location of the passed variable and parameter is the same. Therefore, any change to the parameter also reflects in the variable inside its parent function. Now we will see how parameters can be passed by reference:

Example 5: A function that adds 1 to a given value without returning the new value.

```
#include <iostream.h>
void addOne(int &ref)
{
    ref = ref + 1;
}
int main()
{
    int value;
    cout<<"Enter an integer number to add (1) to it:";
    cin>>value;
    cout << "value = " << value << '\n';
    addOne(value);
    cout << "value after adding one = " << value << '\n';
    return 0;
}
```

A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\loaya\Downloads\V". The prompt displays the following text: "Enter an integer number to add (1) to it:3", "value = 3", "value after adding one = 4", and "Press any key to continue". The user has entered the number 3, and the program has output the value and its incremented version.

So what is a Pass by Reference?

When a variable is passed as a reference to a function, the address of the variable is stored in a pointer variable inside the function. Hence, the variable inside the function is an alias for the passed variable. Therefore, any operations performed on the variable inside the function will also be reflected in the calling function.

- This ability to reflect changes could return more than one value by a function.
- Also, a void function could technically return value/s using this method.

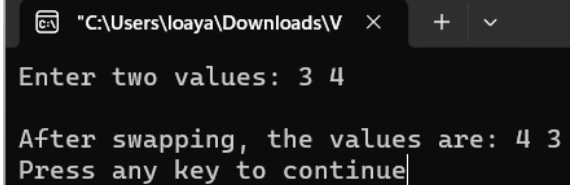
The **&** (address of) operator denotes values passed by pass-by-reference in a function. Next example is another C++ program to implement pass-by-reference:

Example 6: Here is another example of passing parameters by reference, here the function is swapping the values of two variables at their actual location in memory.

```
#include <iostream.h>

void SwapValue(int &a, int &b)
{
    int t = a;
    a = b;
    b = t;
}

int main()
{
    int a, b;
    cout<<"Enter two values: ";
    cin>>a>>b;
    SwapValue(a, b);
    cout<<"\nAfter swapping, the values are: "<<a<<" "<<b<< endl;
    return 0;
}
```



```
"C:\Users\loaya\Downloads\V x + v
Enter two values: 3 4
After swapping, the values are: 4 3
Press any key to continue
```

Example 7: Now, consider the following function, which will take an array as an argument along with another argument, and it will return average of the numbers passed through the array:

```
#include <iostream.h>

// function declaration:
double getAverage(int arr[], int);

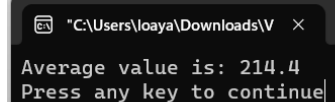
int main ()
{
    // an int array with 5 elements.
    int balance[5] = {1000, 2, 3, 17, 50};

    double avg;

    // pass pointer to the array as an argument.
    avg = getAverage( balance, 5 ) ;

    // output the returned value
    cout << "Average value is: " << avg << endl;

    return 0;
}
```



```
"C:\Users\loaya\Downloads\V x +
Average value is: 214.4
Press any key to continue
```

```
double getAverage(int arr[], int size)
{
    int i, sum = 0;
    double avg;
    for (i = 0; i < size; ++i) {
        sum += arr[i];
    }
    avg = double(sum) / size;
    return avg;
}
```

Assignment No. 2: (Due date: 15/1)

Q1: Write a C++ program to calculate the Bth power of A for non-negative value of B using a user-defined function (Without using the library `pow(A,B)` function.)

$$A^B = A * A * A * A * \dots * A$$

Q2: Write a C++ program to read a sequence of integers, terminated by a negative number. When a positive number is read, display asterisks (*) using a function as many as the input number passed as a parameter, and display the word 'ZERO' if the input number is 0.

Example: 3

 5

 0
 ZERO
 -1

Q3: Write a C++ program that reads a collection of car speeds in an array. Pass the array to a function and count and print the number of speeds less or equal to 55 km/h, and the number exceeds 55 km/h.

Q4: Write a C++ program to find the summation from 1 to x, for n input values using a function to calculate the sum.

Example: How many numbers? 2
 Enter #1: 3
 sum=1+2+3 = 6
 Enter #2: 7
 sum= 1+2+3+4+5+6+7 = 28

Q5: Write a C++ program to find the partial sum for each of the following series, use functions to calculate factorial and power:

$$\text{a.) } x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!} \qquad \text{b.) } x - \frac{x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} \dots \pm \frac{x^n}{n!}$$

Q1: Write a program to input two numbers A and B and see if A is divisible by B.

```
# include <iostream.h>

void main()
{ int a,b;

  cout<<"Enter two numbers:";
  cin>>a>>b;

  if (a%b==0)
    cout<<b<<" divides "<<a<<endl;
  else
    cout<< b<< " does not divide "<<a<<endl;
}
```

Q2: Write a C++ program to find and display the summation of the first N squares of integer numbers.

```
# include <iostream.h>

void main()
{ int N, sum=0;

  cout<<"Enter a positive integer:";
  cin>>N;

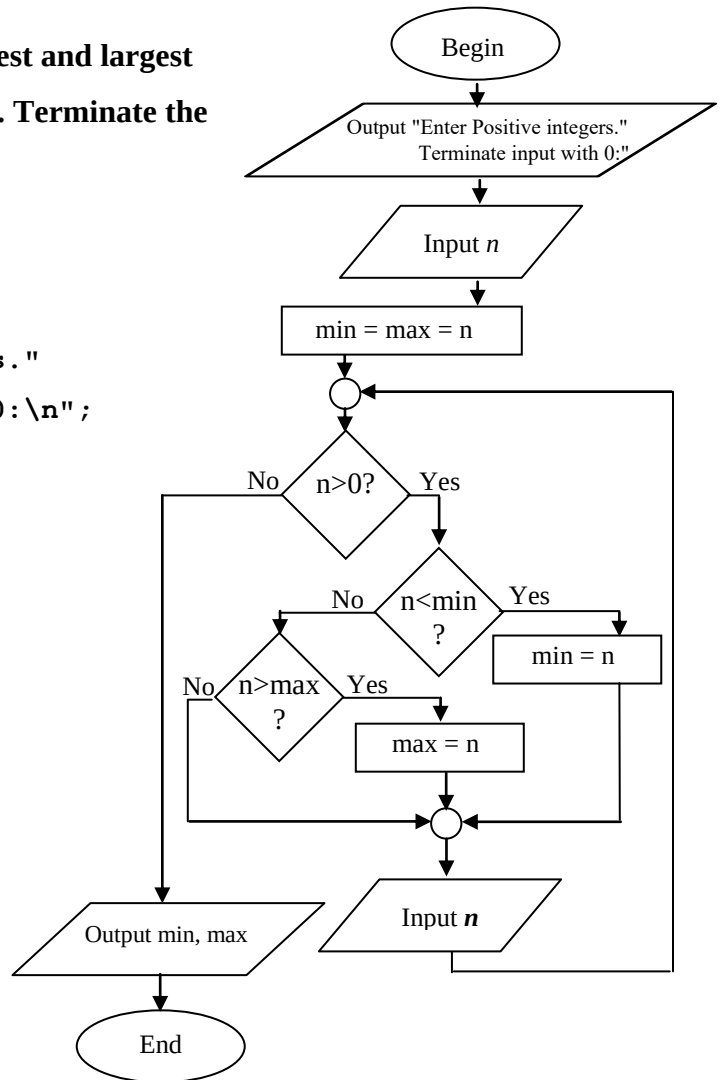
  for (int i=1; i<=N; i++)
    sum+=i*i;

  cout<<"The sum of the first "<<N
    <<" squares is "<< sum<<endl;
}
```

Q3: Write a C++ program to find the smallest and largest value of a series of positive entered integers. Terminate the input with 0.

```
#include <iostream.h>

void main()
{
    int n, min, max;
    cout<<"Enter Positive integers."
    <<" Terminate input with 0:\n";
    cin>>n;
    for(min=max=n; n>0; )
    {
        if (n<min)
            min=n;
        else if (n>max)
            max =n;
        cin>>n;
    }
    cout<<"min = "<< min
    <<" and max = "
    <<max<<endl;
}
```



Q4: Write a C++ program to find the average of a series of positive entered integers. Terminate input with 0.

First Method:

```
#include <iostream.h>

void main()
{
    int n, count = 0, sum = 0;
    cout<<"Enter Positive integers."
    <<" Terminate input with 0:\n";
    for(;;)
    {
        cout<<"\t"<<count+1<<" : ";
        cin>>n;
        if (n == 0) break;
    }
}
```

Exercises on loops and how to make decisions in C++ programming language

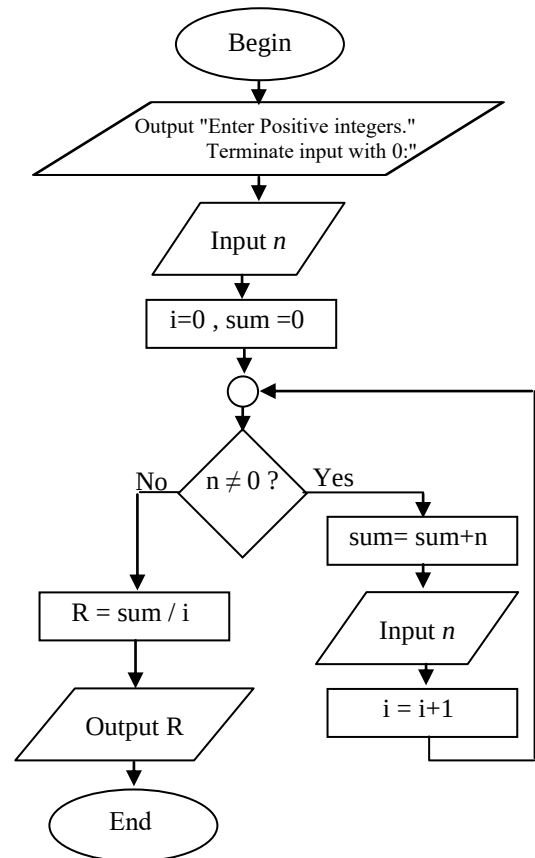
```
    ++ count;
    sum +=n;
}
cout<<"The average of the "<<count
    <<" numbers is "<<float(sum)/count<<endl;
}
```

Second Method:

```
#include<iostream.h>
#include <conio.h>
void main()
{clrscr();
 int n,i=0,sum=0;
 cout<<"Enter positive integers,"
    <<" terminat with 0:";
 cin>>n;
 while(n != 0)
 { sum +=n;

    cin>>n;

    i++;
 }
 cout<<"Average = "<< float(sum)/i;
}
```



Try to answer these Questions:

Q5: Write a program to calculate the B power of A for non-negative value of B.

$$A^B = A \times A \times A \dots \times A \quad \text{Without using the } \text{pow}(A,B) \text{ function.}$$

Q6: Write a program to read a sequence of integers, terminated by a negative number. When a positive number is read, display asterisks (*) as many as the input number, and display the word 'ZERO' if the input number is zero.

Q7: Write a program that reads a collection of car speeds as long as the speeds do not exceed 100 km/h. Count and print the number of speeds less or equal to 55 km/h, the number exceeds 55 km/h, and the total of speeds.

Q8: Write a program to find the summation from 1 to x, for 20 values.

Q9: Write a program in C++ to calculate the series $(1) + (1+2) + (1+2+3) + (1+2+3+4) + \dots + (1+2+3+4+\dots+n)$.

Sample Output:

Input the value for nth term: 5

$$1 = 1$$

$$1+2 = 3$$

$$1+2+3 = 6$$

$$1+2+3+4 = 10$$

$$1+2+3+4+5 = 15$$

The sum of the above series is: 35

C++ PROGRAMMING LANGUAGE

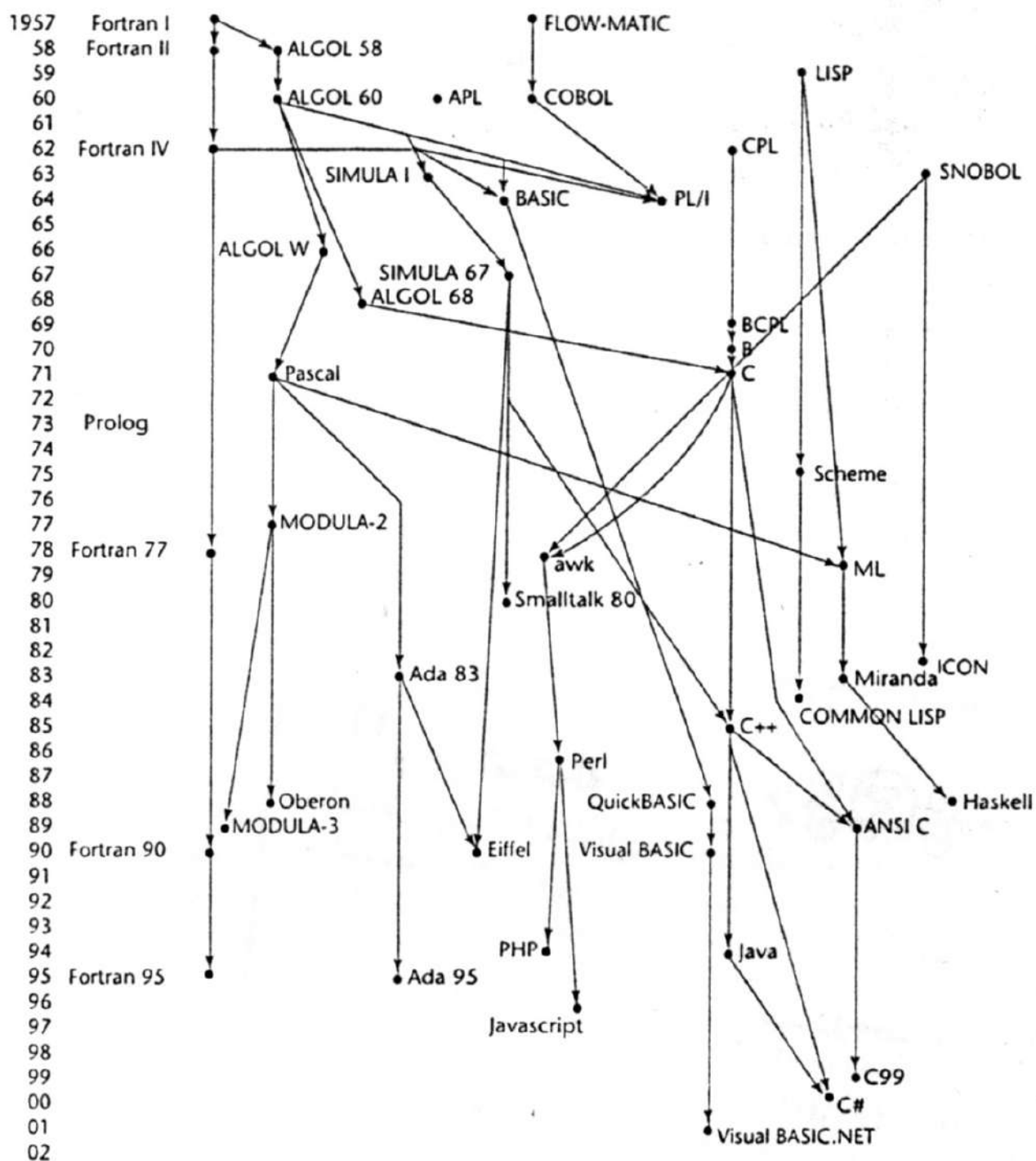
BY
DR. LOAY AL-SAFFAR
EETC - MTU
BAGHDAD – IRAQ

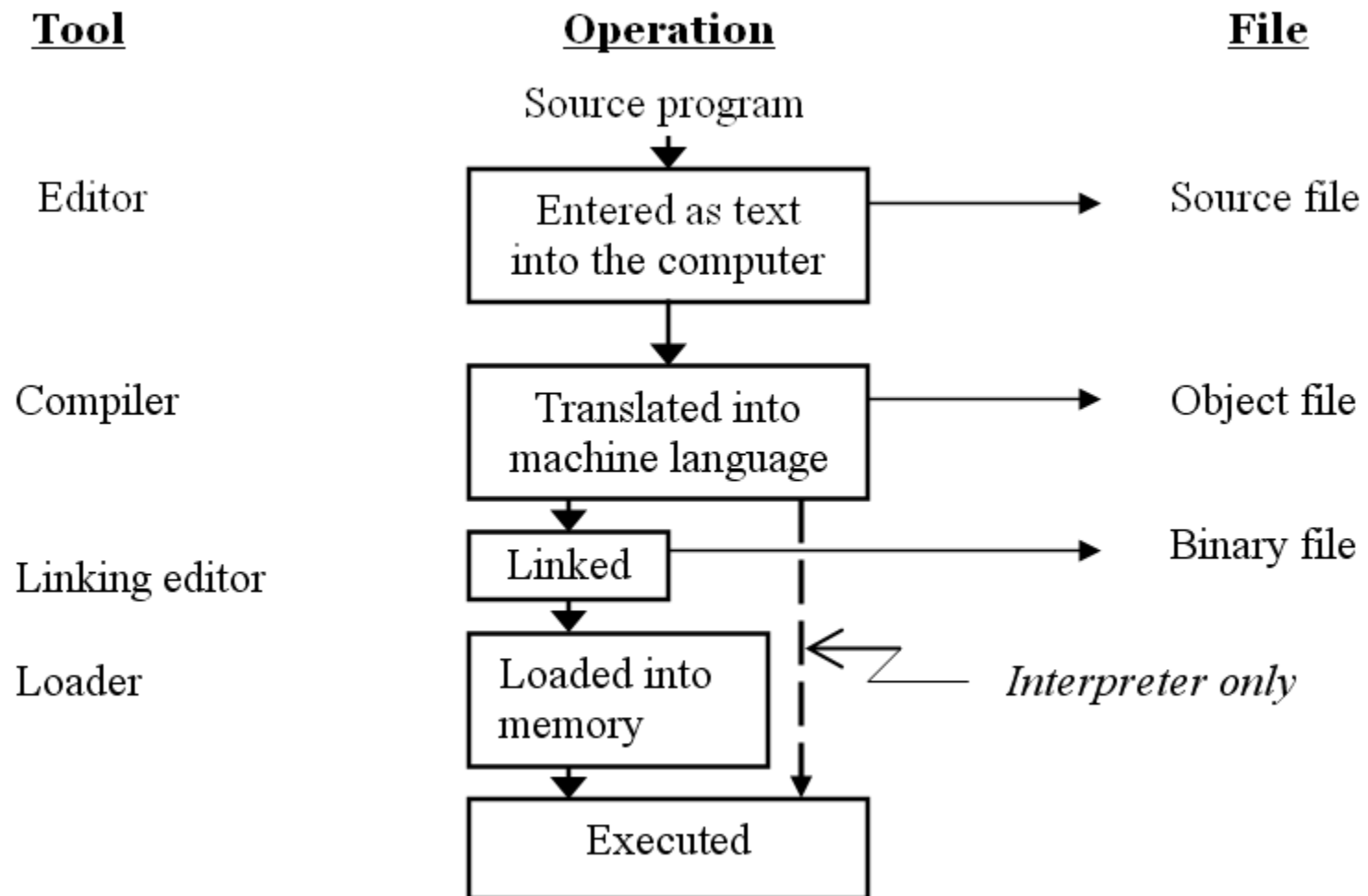
HISTORICAL VIEW

C++ is an evolution of the language C that was developed at AT & T Bell laboratories in the early 1980s by *Bjarne Stroustrup*.



C++ has become the language of choice of many software professionals, engineers, and scientists not only because it has powerful commands & data structures, but also because it can easily be used for system-level operations.





Dealing with program errors

Errors are a fact of programming life. All programmers make mistakes, so don't feel bad when errors occur.



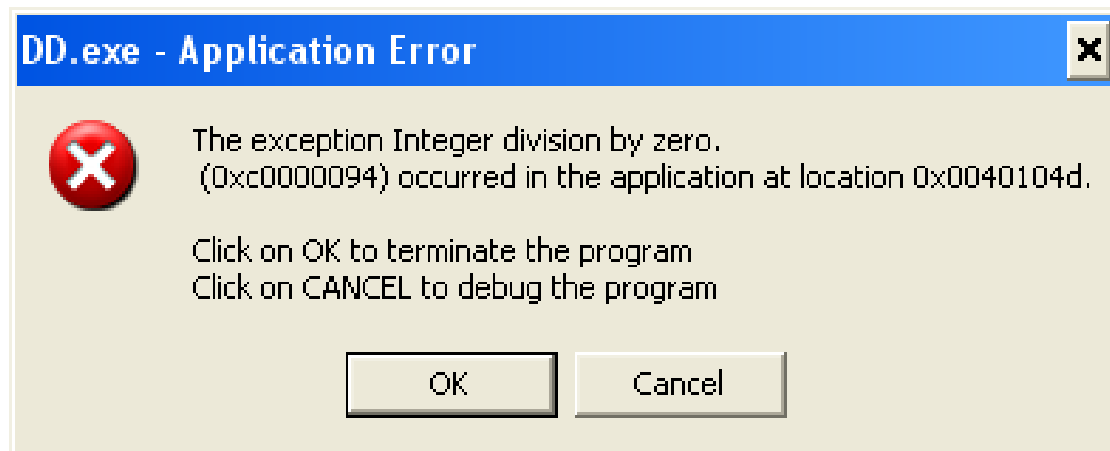
There are three fundamental types of errors:

Compile-time errors (*syntax errors*), a syntax error wastes computing time as well as programmers time, and it is preventable.



..Continue

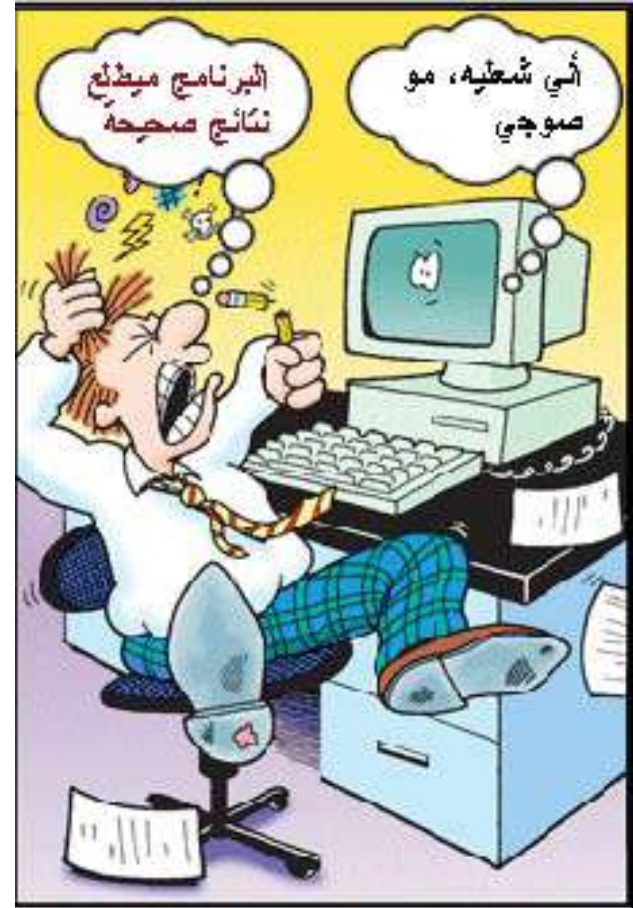
Run-time errors, are errors that occurs during the execution of a program, they are usually harder to detect than syntax errors. if the error stops the execution of the program, we say that the program "**crashed**" or "**abnormally terminated**", for example, division by zero.



Continue..

Logic errors, occurs when the program runs to completion but the results are incorrect. Somehow the program does not work correctly. In programming terminology, the program contains a ***bug***.

The programmer must *debug* the program, which means finding and correcting the error.



The First Computer Bug

92

9/9

0800 Antan started

1000 stopped - antan ✓

1300 (033) MP-MC { 1.2700 9.037 847 025
~~2.130476415~~ 9.037 846 795 convd
 (033) PRO 2 2.130476415
 convd 2.130676415

Relays 6-2 in 033 failed special speed test
 in relay 10.00 test.

Relays changed

1100 Started Cosine Tape (Sine check)

1525 Started Multi-Adder Test.

1545

Relay #70 Panel F
 (moth) in relay.

First actual case of bug being found.

1700 Antan started.

1700 closed down.



Grace Hopper (1906-1992), (PhD in Mathematics, 1934), found the first computer bug while working in a temporary World War II building at Harvard University on the Mark I computer, 9 September 1945

YOUR FIRST C++ PROGRAM

```
#include <iostream.h>
void main() //program heading
{ //begin
    cout << "Hello World!\n";
} //end
```

output: **Hello World!**

COMMENTS

- Any text from the symbols `//` until the end of the line is ignored by the compiler. This facility allows the programmer to insert Comments in the program for documentary purpose. Comments in the program for documentary purpose. You can also use the symbols `/* */` to write a Comment between them (known as C style.)
- Example: `/* My first program */`

CASE SENSITIVITY

- C++ is case-sensitive.
- In other words, uppercase and lowercase letters are considered to be different.
- A variable named **age** is different from **Age**, which is different from **AGE**.

Variables



- **A variable is a named data storage location in your computer's memory.**
- `int a=3;`

In C++, variable names must stick to the following rules:

The name can contain letters, digits, and the underscore character (`_`). The first character of the name must be a letter. C++ keywords can't be used as variable names.

examples:

Percent
sum1
sum_1
avg1a

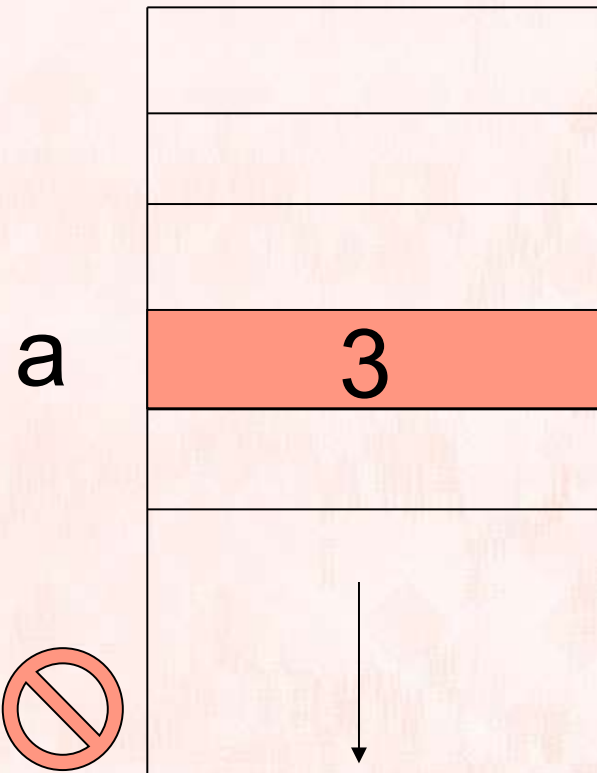
1a
me#you
int



Spaces are not allowed in variable names:



Ahmed 2

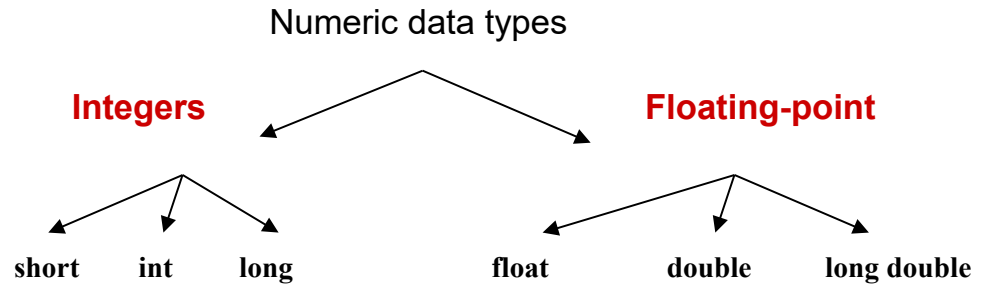


Variable Types

<i>Type</i>	<i>Size</i>	<i>Values</i>
unsigned short	2 bytes	0 to 65,535
short	2 bytes	-32,768 to 32,767
unsigned long	4 bytes	0 to 4,294,967,295
long	4 bytes	-2,147,483,648 to 2,147,483,647
int	2 bytes	-32,768 to 32,767
unsigned int	2 bytes	0 to 65,535
char	1 byte	256 character values
float	4 bytes	1.2e-38 to 3.4e38
double	8 bytes	2.2e-308 to 1.8e308

Note : int on a Visual C++, occupies 4 bytes.

Numeric Data Types



Floating point

float *6 digits of precision*, maximum exponent = 38, maximum value = 3.402823e+38

double *15 digits of precision*, maximum exponent = 308, maximum value = 1.797693e+308

```
int count, number, start;    /* three integer variables */
```

```
float percent, total;        /* two float variables */
```

```
typedef int integer; // create a new name for an existing data type
```

```
integer count;
```

YOUR SECOND C++ PROGRAM

```
// by programmer name, October 2023
// this program computes
// the distance between two points.
#include <iostream.h>
#include <math.h>
void main( )
{ // define and initialize variables.
  double x1=1,y1=5,x2=4,y2=7,side_1,side_2,distance;

  // compute
  side_1=x2-x1;
  side_2=y2-y1;
  distance = sqrt(side_1*side_1 + side_2*side_2);

  // print distance.
  cout <<"the distance between the two points is"
    << distance<< " cm."<< endl;
}
```

Boolean Data Type

(bool used in VC++ and its not available for Turbo C++)

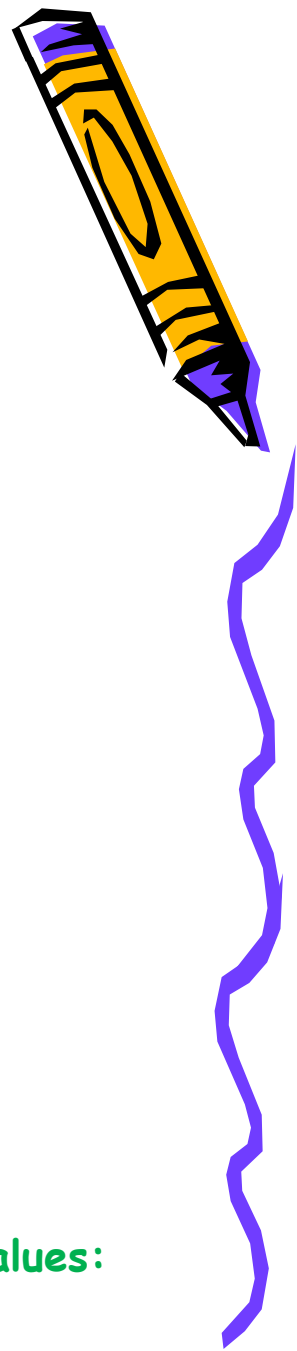
```
#include "iostream.h"
```

```
int main()  
{  
    bool f;  
    f= false;  
    cout<<f;  
    return 0;  
}
```

Output will be 0

We can use the following rules to deal with true and false values:

- A value of zero represents false.
- Any nonzero value represents true.



Constasnts

literal constant : **int myAge = 39**

//39 is a literal constant

symbolic constant :

1- #define studentsPerClass 15

2- const int studentsPerClass = 15;

Statements

Expression statements : **m = x+y;**

Compound statements, also called a block:

```
{ a=b+c;  
  x=x*x;  
  y=a+x;  
}
```

Control statements : control the program flow:

ex.:

```
  if (condition becoming true)  
while(condition becoming true)
```

Standard Input and Output

- `cout << "A Program by loay \n"`
`<< "enter two integers:" << endl;`

Separate output with the
insertion operator

- `cin >> a >> b;`

Separate input with the
extraction operator

Standard Input and Output

- `cout << "A Program by loay \n"`
`<< "enter two integers:" << endl;`
- `cin >> a >> b;`

Only used with cout

Standard Input and Output

- `cin >> "enter two integers:" >> a >> b;`
- `cin >> a, b;`
- `cin >> "a";`

SOME C++ OPERATORS

Arithmetic Operators:

The five operators (+, -, *, /, %)

Assignment Operator: The symbol =

x=y=z=0

Increment and Decrement Operators:

w = ++x - y; // is equivalent to **x = x+1; w = x-y;**

w = x++ - y; //is equivalent to **w = x-y; x = x+1;**